

**HELLENIC MEDITERRANEAN UNIVERSITY
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
FACULTY OF ENGINEERING**

**A FINAL REPORT
ON
MACHINE LEARNING-BASED ANALYSIS AND DETECTION OF PHISHING
WEBSITES USING FEATURE AND CLASSIFICATION MODELS**

**REGARDING THE SUBJECT:
MACHINE LEARNING**

**BY
KONSTANTINIDOU ELENI (TH20025)**

**SUPERVISED BY:
Dr. KARAMPIDIS KONSTANTINOS**

HERAKLEIO, 2025

ABSTRACT

This research is dedicated to the development of a robust machine learning framework for the detection of phishing websites, one of the most current cybersecurity threats. The study uses a dataset from the UC Irvine Machine Learning Repository and carries out a systematic approach that includes exploratory data analysis, feature selection, and classification using various machine learning models. It identified the most important features indicative of phishing activities and applied state-of-the-art techniques such as Sequential Feature Selection and dimensionality reduction using autoencoders to improve model performance.

The analysis was done using classifiers like logistic regression, support vector machines, decision trees, random forests, and XGBoost, while neural network models included Multilayer Perceptrons. Other insights on data structure were obtained with k-means and agglomerative clustering. Among the evaluation metrics considered were accuracy, precision, recall, F1-score, and AUC-ROC, reflecting the performance of the proposed models, while XGBoost and neural networks performed better.

Specifically, feature engineering coupled with robust classifiers enhances the interpretability of phishing detection. These contributions form part of the improvement in adaptive and effective systems to meet dynamic changes in phishing tactics, further improving cybersecurity for various domains.

Keywords: phishing websites, machine learning, classification models, classification algorithms, clustering techniques, feature extraction, neural networks, evaluation metrics

1. Introduction

The rapid growth of the internet has transformed in many ways commerce and communication, but it also has led to the emergence of numerous cyberthreats, including phishing. Phishing refers to the practice of tricking Internet users, via deceptive email messages or websites, into revealing personal or confidential information, such as passwords and financial IDs, which can then be used illicitly [1]. Despite the significant developments in the cybersecurity field, phishing attacks are still constant and continue to evolve; thus, strong detection mechanisms are required for users' and systems' protection.

The main problem addressed in the research work is the correct and precise identification of phishing websites. Most of the conventional detection measures, such as blacklisting and heuristic-based approaches, frequently are unable to keep up to the dynamic tactics of the attackers. Furthermore, these methods have a high rate of false positive and therefore, there is an urgent demand for the development of machine learning-based solutions that can adapt to evolving attack patterns and improve detection accuracy.

The goal of this study is to develop and evaluate machine learning models for the detection of phishing websites using a dataset from the UC Irvine Machine Learning Repository. This dataset includes many important features that capture the characteristics of both legitimate and phishing websites, hence providing a strong foundation for testing and training classification algorithms. Also, it aims at the exploration of feature selection techniques for identifying the most relevant attributes that help in effective detection and check the performance of various machine learning classifiers.

2. Related Work

The detection of phishing sites has been among the most significant research areas in cybersecurity owing to the continually changing sophistication by attackers. Traditional approaches normally adopted include blacklisting and heuristic-based methods, which cannot adapt with the dynamics of phishing tactics and, therefore, are mostly related to very high false-positive rates. In recent times, developments within machine learning have opened ways toward robust mechanisms for phishing detection.

Mohammad et al. (2012) targeted feature-based analysis for phishing detection and pointed out some important features, such as address bar-based features, domain-based features, and HTML content features, which are highly indicative of phishing websites [2]. Similarly, Cheng Fan et al. (2021) reviewed data preprocessing techniques that enhance the performance of machine learning models in identifying phishing websites [3]. Recent works by Bu and Cho, 2021, emphasized the use of

convolutional autoencoders for zero-day phishing detection and showed the capability of deep learning models in detecting sophisticated threats [4].

ML algorithms, for example, logistic regression, decision trees, and ensemble methods like random forests, have shown a lot of promise in classification. The neural network architecture, including MLPs, outperforms the traditional models because of their capacity to learn higher-order patterns from data with great complexity [5][6]. These approaches often integrate dimensionality reduction techniques like PCA to facilitate interpretability and enhancement in model performance [7].

Despite all these developments, several challenges in machine learning-based phishing detection are yet to be fully overcome. The existing models, for example, rely excessively on static feature sets, thus affecting their applicability to newer, evolving phishing strategies. Additionally, most works bypass the interpretability aspect of machine learning models—a crucial factor for many real-world applications, since decision transparency plays a very significant role. High-accuracy results for ensemble and neural network models incur high computational cost and thus have less viability in resource-constrained environments.

One of the most important challenges in this research and largely faced by the field is the lack of reliable training datasets. Several works have proposed different methods for phishing detection, but there has not been any consensus on what features essentially describe phishing websites. Consequently, it has been challenging to shape a dataset that comprehensively covers all possible features. The absence of such standardized datasets seriously hinders the development of robust models and reproducibility, one of the key barriers to benchmarking and comparative analysis [8].

Another notable gap is in the evaluation metrics used. Many studies use accuracy without properly discussing the trade-offs between false positives and false negatives. This further complicates cross-study comparisons due to the lack of a standardized benchmark for datasets and performance metrics.

This work tries to bridge these gaps by availing a complete feature set from the UCI Machine Learning Repository [9], advanced feature selection techniques such as Sequential Feature Selection, and several classifiers including XGBoost and support vector machines. Model performance will further be gauged on the basis of robust metrics that include AUC-ROC and F1-score, which would ensure balanced performance evaluation. Besides, the autoencoder neural networks for dimensionality reduction provide a scalable approach to high-dimensional datasets.

This research contributes not only to the enrichment of previous studies but also to the introduction of innovations in feature selection and model evaluation, opening up wider perspectives toward more adaptive, interpretable, and efficient phishing detection systems.

3. Dataset Description

3.1. Overview of the Dataset

The dataset used for this analysis was retrieved from the UCI Machine Learning Repository, a collection of databases and data generators that are used by the machine learning community for the empirical analysis of machine learning algorithms [2]. This particular dataset is used for detecting phishing websites and contains features designed to identify suspicious website characteristics. The features are categorized into four main types [10] :

- Address Bar-Based Features, which include features related to the content and structure of the URL
- Abnormal-Based Features, which include features that indicate inconsistencies or unusual behaviors
- HTML and JavaScript-Based Features, which include features regarding manipulations and scripts affecting user interactions and webpage behavior
- Domain-Based Features, which include features that assess the legitimacy and reputation of the domain

Below is a detailed explanation of each feature:

Category	Attribute	Value Range	Meaning
Address Bar-Based	having_IP_Address	{-1,1}	-1 for phishing, 1 for legitimate
-/-	URL_Length	{1, 0, -1}	1: Legitimate (short), 0: Suspicious (medium), -1: Phishing (long)
-/-	Shortining_Service	{1, -1}	-1: Phishing, 1: Legitimate
-/-	having_At_Symbol	{1, -1}	-1: Phishing, 1: Legitimate
-/-	double_slash_redirecting	{-1, 1}	-1: Phishing, 1: Legitimate
-/-	Prefix_Suffix	{-1, 1}	-1: Phishing, 1: Legitimate
-/-	having_Sub_Domain	{-1, 0, 1}	1: Legitimate (no subdomains), 0: Suspicious (one subdomain), -1: Phishing (multiple subdomains)

-/-	SSLfinal_State	{-1, 0, 1}	1: Legitimate, 0: Suspicious, -1: Phishing
-/-	Domain_registration_length	{-1, 1}	-1: Phishing (short period), 1: Legitimate (long period)
-/-	Favicon	{1, -1}	-1: Phishing, 1: Legitimate
-/-	port	{1, -1}	-1: Phishing (non-standard ports), 1: Legitimate
-/-	HTTPS_token	{1, -1}	-1: Phishing, 1: Legitimate
Abnormal Based	Request_URL	{1, -1}	-1: Phishing, 1: Legitimate
-/-	URL_of_Anchor	{1, 0, -1}	1: Legitimate, 0: Suspicious, -1: Phishing
-/-	Links_in_tags	{1, 0, -1}	1: Legitimate, 0: Suspicious, -1: Phishing
-/-	SFH	{1, 0, -1}	1: Legitimate, 0: Suspicious, -1: Phishing
-/-	Submitting_to_email	{1, -1}	-1: Phishing, 1: Legitimate
-/-	Abnormal_URL	{1, -1}	-1: Phishing, 1: Legitimate
HTML & JavaScript Based	Redirect	{1, 0}	0: Phishing (multiple redirects), 1: Legitimate (single or no redirect)
-/-	on_mouseover	{1, -1}	-1: Phishing, 1: Legitimate
-/-	RightClick	{1, -1}	-1: Phishing, 1: Legitimate
-/-	popUpWidnow	{1, -1}	-1: Phishing, 1: Legitimate
-/-	Iframe	{1, -1}	-1: Phishing, 1: Legitimate
Domain Based	age_of_domain	{1, -1}	-1: Phishing (young domain), 1: Legitimate (older domain)
-/-	DNSRecord	{1, -1}	-1: Phishing (no record), 1: Legitimate
-/-	web_traffic	{1, 0, -1}	1: Legitimate, 0: Suspicious, -1: Phishing
-/-	Page_Rank	{1, -1}	-1: Phishing, 1: Legitimate
-/-	Google_Index	{1, -1}	-1: Phishing, 1: Legitimate

-/-	Links_pointing_to_page	{1, 0, -1}	1: Legitimate, 0: Suspicious, -1: Phishing
-/-	Statistical_report	{1, -1}	-1: Phishing, 1: Legitimate
	Result (target variable)	{1, -1}	-1: Phishing, 1: Legitimate

Table 1: Key Features of Phishing Website Detection

Each of the features outlined above is governed by specific rules that provide clear and explicit criteria for determining whether a website is classified as phishing or legitimate. These rules serve as the foundation for systematically analyzing website behavior and characteristics

3.2. Phishing Detection Rules for Each Feature:

Attributes	Description	Rule
having_IP_Address	Checks if an IP address is used in the URL instead of a domain name.	If the domain part has an IP Address is classified as "Phishing", otherwise the URL is legitimate
URL_Length	Measures the length of the URL to hide suspicious parts.	If URL length is shorter than 54 characters the feature is legitimate, if the length is somewhere between 54 and 75 the feature is suspicious, otherwise if the URLs length is greater than 54 characters the feature is considered "Phishing".
Shortining_Service	Detects shortened URLs that redirect to longer ones.	If TinyURL then it is considered phishing, otherwise it is legitimate
having_At_Symbol	Checks for the presence of the "@" symbol in the URL.	URLs having the symbol "@" are considered phishing, otherwise they are legitimate
double_slash_redirecting	Analyzes the position of "/" in the URL.	If the position of the last occurrence of "/" in the URL is greater than seven then it's considered phishing, otherwise it's legitimate

Prefix_Suffix	Checks for the use of dashes in domain names.	If the domain name part includes a dash (-) symbol it is considered phishing, otherwise it is legitimate
having_Sub_Domain	Counts the number of dots in the URL.	If the number of dots in domain part is equal to one the URL is classified as legitimate, if the number of dots is equal to two then the URL is suspicious. Otherwise, if the number of dots are greater than two it is classified as phishing
SSLfinal_State	Verifies HTTPS usage and certificate trust level and age.	If there's use of 'https' and the issuer is trusted and the age of certificate is greater than a year then the feature is legitimate, if there's use of 'https' and the issuer is not trusted then the feature is suspicious. Otherwise it's phishing.
Domain_registration_length	Analyzes the registration period.	If the domain expires on less than a year then it is considered phishing, otherwise it is legitimate
Favicon	A favicon is a small graphic icon associated with a webpage. It is displayed in the browser's address bar.	If the favicon is loaded from an external domain then it's considered phishing, otherwise it's legitimate
port	Checks if the URL uses non-standard ports (e.g., for HTTP or HTTPS services).	If non-standard ports are open, classify as "Phishing". Otherwise it is legitimate.
HTTPS_token	Verifies if the term "HTTPS" is misleadingly added to the domain name to trick users.	If "HTTPS" Token appears in the domain name but is unrelated to secure protocols, classify as "Phishing". Otherwise it is legitimate.
Request_URL	Examines external objects (images, videos) loaded from other domains.	If the percentage (%) of Request URL is smaller than 22% then it is legitimate, if it is between 22% and 61% then it's suspicious. Otherwise, the feature is considered phishing.
URL_of_Anchor	Checks anchor <a> tags linking to different domains.	External anchors or empty links are "Phishing". In more detail, if the percentage of URL of Anchor is smaller than 31% then the URL is

		legitimate. If the percentage value is between 31% and 67% then the URL. Is suspicious. Otherwise it is considered phishing.
Links_in_tags	Analyzes tags linking to external domains.	If the % of Links in "< Meta >", "<Script>" and "<Link>" is smaller than 17% then the tag is considered legitimate. If the % of Links in "< Meta >", "<Script>" and "<Link>" is greater than 17% And smaller that 81% then it is considered suspicious. Otherwise it is considered phishing.
SFH	Checks if the form handler is blank or mismatches the webpage domain.	If SFH is "about: blank" Or it is Empty then the SFH is considered phishing. If the SFH regers to a different domain then it is suspicious. Otherwise it is considered legitimate.
Submitting_to_email	Detects personal information submitted to email addresses.	If there is presence of "mail()" or mailto: function to submit user information then indicates phishing. Otherwise it is considered legitimate.
Abnormal_URL	Validates URL identity using WHOIS database.	If the Host name is not included in the URL then the URL is considered phishing. Otherwise the URL is legitimate.
Redirect	Counts the number of website redirections.	If a website has been redirected less than one time then it is legitimate. If a a website has been redirected more than two but less than four times it is considered suspicious. Otherwise, if a website has been redirected more than four times it is considered phishing.
on_mouseover	Checks for fake URLs in the status bar via JavaScript.	If the "onMouseOver" event changes the status bar then this indicates phishing. Otherwise, if it doesn't change status bar is cosidered legitimate.
RightClick	Detects JavaScript disabling right-click functionality.	If the right-click is disabled it indicates phishing. Otherwise it is legitimate.

popUpWidnow	Identifies pop-ups asking for personal information.	If a pop-up window contains text fields it indicates phishing. Otherwise it is legitimate.
Iframe	Checks for invisible <iframe> elements.	If there is use of iframe it indicates phishing. Otherwise it is considered legitimate.
age_of_domain	Measures domain age from WHOIS data.	If the age of domain is greater than 6 months it indicates legitimacy. Otherwise indicated phishing.
DNSRecord	Checks if the domain has a valid DNS record.	If there is no DNS record for the domain it is considered phishing. Otherwise it is legitimate.
web_traffic	Measures website traffic and Alexa ranking.	If the Website Rank is smaller than 100.000 it is legitimate. If it is greater than 100.000 is suspicious. Otherwise it is considered phishing
Page_Rank	Measures importance on a scale of 0 to 1.	If the PageRank is smaller than 0.2 it indicates phishing. Otherwise it is legitimate.
Google_Index	Checks if the website is indexed by Google.	If a webpage is indexed by Google then is considered legitimate. Otherwise it is considered phishing
Links_pointing_to_page	Counts the number of backlinks to the webpage.	If the number of Link Pointing to the webpage is equal to zero it indicates phishing. If the number of Links pointing to the webpage is between zero and two it is suspicious. Otherwise it is legitimate.
Statistical_report	Uses statistical reports from databases tracking phishing activities.	If the Host belongs to Top phishing Ips or Top phishing Domains it indicates phishing. Otherwise it is legitimate.

Table 2: Phishing Detection Rules and Criteria

4. METHODS

4.1. Design of the Study

The design of this study follows a standard systematic machine learning pipeline in trying to resolve the challenge of phishing website detection. It encompasses four stages of procedures, namely Exploratory Data Analysis (EDA), data pre-processing,

training of different models, and evaluation. Each step of these stages has been carefully designed with considerations to ensure reliability and accuracy.

The exploratory data analysis helps us gain insights into the dataset to understand its structure, quality and key patterns. Data preprocessing includes cleaning and normalizing the dataset for the input data to become prepared for model training. These are followed by the training of various machine learning algorithms, which include MLP, XGBoost, Support Vector Machine (SVM), and Perceptron. Feature representations are enhanced with the help of an autoencoder as a dimensionality reduction technique. Lastly, different performance metrics using cross-validation are performed to see that all the trained models generalize well. This systematic approach ensures that the models are effective as well as interpretable for phishing detection.

More specifically, each stage of the process is further analyzed below.

4.1.1. Exploratory Data Analysis (EDA):

The Exploratory Data Analysis (EDA) is an important approach that enhances accessibility [11] and helps in identifying the main characteristics, structure and key patterns of a dataset. This method involves examining distributions, relationships and potential issues and thereby provides a better understanding of the data, with statistical graphs and other visualization methods. This EDA process began by importing the necessary libraries and loading and preparing the dataset for analysis.

4.1.1.1. Dataset Inspection

Firstly, the dataset “phishing_dataset.arff”, was loaded into the environment using the Pandas library. The data file was observed to be a comma-separated, therefore we specified the necessary parameter while reading the file. In order to make the dataset more interpretable, we assigned column names to each feature, that represent the attributes of the phishing URLs as well as the final classification result. After that, we displayed the data to check if all the columns and their features are added correctly. We proceed with the analysis by gathering general information about the dataset, including its shape, the total number of instances, the types of features, and the nature of the data it contains.

4.1.1.2. Feature Distributions

To understand better the structure of the dataset and explore the distribution of classes as well as to detect the existence of any outliers several distribution plots were performed.

Principal Component Analysis (PCA): Principal Component Analysis (PCA) is a dimensionality reduction technique that simplifies the data while retaining as much variability (information) as possible. PCA reduces the data to 2D for visualization, making it easier to understand the relationships between classes [8]. This visualization helps to understand whether the classes, phishing or non-phishing, are linearly separable and whether clusters naturally form in the dataset. In our analysis, the PCA was applied to the feature space, excluding the target column, and the output is a scatter plot that shows the data points distribution in the reduced two-dimensional PCA space.

Data Distribution for the Target Variable: As for the distribution diagram of the target variable “Result”, a bar chart was plotted. The target variable in the dataset has two possible values:

- the value 1 that represents the legitimate URLs and
- the value -1 that represents the phishing URLs.

The objective of visualizing the target variable’s distribution is to assess the balance of the dataset.

Data Distribution Across Features: In order to gain insights into the characteristics of each feature in the dataset, the distribution across all features was also analyzed. The dataset was iteratively grouped by each feature and the results were visualized via histograms.

Feature-Value Contribution Across Classes: An important step in exploratory analysis is calculating the contribution of each feature’s values to the target classes (-1 for phishing and 1 for legitimate URLs). This provides insights into how each feature relates to the target variable. For this implementation of this step, for each feature, the dataset was grouped by its unique values and the target variable (“Result”) and then the proportions of each class for each feature value were computed and converted into percentages.

Correlation Matrix Heatmap: In order to examine the connections between the dataset’s numerical properties, a correlation matrix was created. This stage gives information about how linearly related feature pairs are, which might affect feature selection and model performance. In more detail, a correlation matrix is a table that shows the correlation coefficients between the different features of a dataset. Each cell in the table shows the correlation between two features [12].

For the calculation of the correlation, the Pearson correlation coefficient was used to measure the strength of the linear relationships between features. The Pearson correlation coefficient (r) is a statistical measure that ranges from -1 to +1 and is defined as follows [13]:

$$r_{XY} = \frac{n \cdot \sum_{i=1}^n x_i \cdot y_i - (\sum_{i=1}^n x_i) \cdot (\sum_{i=1}^n y_i)}{\sqrt{n \cdot \sum_{i=1}^n x_i^2 - (\sum_{i=1}^n x_i)^2} \cdot \sqrt{n \cdot \sum_{i=1}^n y_i^2 - (\sum_{i=1}^n y_i)^2}} \quad (1)$$

In our analysis, the correlation matrix was computed using the `corr()` function in Pandas which calculates the Pearson correlation coefficient by default. Furthermore, in order to make the correlation matrix more interpretable, a heatmap was generated using Seaborn. The heatmap highlights strongly correlated features (both positive and negative) and features with negligible correlation.

Most Relevant Feature Selection: The feature-target variable correlation was analyzed in order to identify the most relevant features for model development. The main purpose of this step is to identify, in order of importance, those features with the highest magnitude of relationship with the target variable, whether positive or negative.

To that effect, the correlations have been computed in two ways to give a broader understanding of feature relevance:

- **Absolute Value of Correlation:**

There is the assessment of the first approach, since the correlation coefficients had been analyzed in their absolute value form, focusing mainly on the magnitude of the features with respect to the target variable, without caring about the orientation of the relationship. A bar plot was provided that showed, in sorted form, the most strong features.

- **Actual Correlation Values:**

The actual value of the different correlations was computed based on their sign, considering the direction of the relationship between each feature and the target variable. The results were then visualized using the bar plot created with Matplotlib for better visualization in their actual form.

This dual analysis helps ensure that both the magnitude and direction of relationships are considered for balanced views on feature selection and subsequent model development.

4.1.2. Data Preprocessing

In order to transform raw data into a clean and organized state, data preprocessing is a fundamental step in machine learning operations. This step greatly improves the quality of the data and the precision of the models by addressing problems like missing values, noise, and inconsistencies. Research highlights the

importance of tasks including dimensionality reduction, scaling, and normalization in enhancing classifier performance and generalization [14], [15]. Good preprocessing is essential to strong and dependable predictive systems since it reduces errors and optimizes data for machine learning algorithms.

In our project implementation, this phase involved the following tasks:

Missing Values: Missing values in the dataset can affect negatively the performance of the models. Therefore, it is mandatory to be addressed before training. We handle the missing values by using functions like “isnull()” or “isna()” in pandas. These functions take a scalar or array-like object and indicates whether values are missing [16]. Sometimes, in a dataset, the missing values are defined with a question mark (“?”). In our analysis we also took that into consideration and checked for the existence of a question mark symbol “?” in the given dataset.

Encoding Categorical Features to Numerical Values: The encoding process makes the categorical variables interpretable to the model and since machine learning models generally require numerical inputs, the categorical data must be transformed. However, a thorough inspection of our dataset revealed that all features were already in numerical format. Hence, no additional encoding was required for this analysis.

Splitting the Data: The dataset was divided into training (80%) and test (20%) sets in order to enable model training and evaluation.

Standardization: Features were standardized using StandardScaler from scikit-learn to have zero mean and unit variance. This step is particularly important for models like SVM and MLP, because they present great sensitivity to feature scaling.

Overfit Check: After the implementation of all the models, we proceeded to do a check for overfitting. Overfitting occurs if the model performs significantly better on the training set than on the test set. If there's a large gap between training and test accuracy, the model might be overfitting.

4.1.3. Model Training

As we highlighted before, machine learning algorithms provide a data-driven approach that may successfully differentiate between phishing and authentic websites by spotting intricate patterns in website attributes. The purpose of training machine learning models in this study is to exploit the rich dataset of features and create classifications models that can successfully identify with high accuracy and reliability phishing websites, reduce false positives (misclassifying trustworthy websites as

phishing) and false negatives (failing to identify phishing websites) as well as adjust to invisible data, guaranteeing this way strong performance in practical situations.

4.1.3.1. Feature Selection for Model Training:

Feature selection for training the different models, was initially performed based on the correlation matrix to narrow down the dataset to the most relevant features. The most significant features retrieved from this analysis constituted the first dataset (dataset 1). To expand even more the analysis and enhance model performance and reduce dimensionality, Sequential Feature Selector (SFS) was also employed. Because of its ease of use, interpretability, and effectiveness in binary classification tasks like phishing website detection, logistic regression was selected as the foundational model for Sequential Feature Selector (SFS). It makes it possible to identify the most pertinent predictors by giving features explicit coefficients. It is perfect for the iterative feature selection process, which adds or removes features to optimise performance metrics, because of its computational efficiency [17]. The most significant features retrieved from the SFS constituted the second dataset (dataset 2).

Based on these two datasets, we proceeded to implement the same analysis pipeline, calculating the same metrics and using the same classifying models.

4.1.3.2. Classifying Models Used:

Logistic Regression: A predictive method called logistic regression seeks to create a model that can predict or explain the values of a qualitative target variable (usually binary) based on a collection of quantitative or qualitative explanatory variables [18]. For modelling relationships with outcomes that are not measured on a continuous scale, a crucial prerequisite for linear regression, logistic regression is a great tool. This model is valued for its simplicity, interpretability, and efficiency in estimating the likelihood of class membership. In order to model this likelihood, logistic regression is frequently used in the classification process. In our implementation, we utilized the LogisticRegression class from the scikit-learn library.

K-Nearest Neighbors (KNN): The K-Nearest Neighbors algorithm is a nonparametric and supervised learning algorithm that is used both for classification and regression. The philosophy of this algorithm is that similar data points tend to be located near each other within any feature space. In the case of classification problems, KNN works by labeling a new point with the most majority class out of its surrounding 'k' neighbors, 'k' being the user-provided parameter. This simplicity and effectiveness make KNN a widely used algorithm in various domains, including phishing website detection. [19]

KNN, for phishing website detection, classifies the websites depending on different features, such as URL characteristics, webpage content, and other attributes. It compares the features of the newly occurring websites with those of the previously

known phishing and legitimate websites to identify potential phishing sites. Being non-parametric, KNN is able to adapt to the underlying data distribution without strong assumptions, which is quite helpful in detecting the diverse and evolving tactics employed in phishing attacks. [20]

However, KNN has some disadvantages, mainly concerning computational efficiency. Since the distance between the new data point and all points in the training dataset has to be computed, for large datasets, this may be computationally expensive. Besides, the performance of the algorithm is sensitive to the choice of 'k' and the distance metric used. Despite these challenges, with proper tuning of parameters and optimization techniques, KNN remains one of the important weapons in the arsenal of machine learning algorithms for such tasks as phishing detection. [19].

Decision Tree: This approach uses a tree-like decision model, with each internal node standing for an attribute test, each branch for the test's result, and each leaf node for a class label [21]. Because of their interpretability and ease of use, decision trees are helpful for comprehending how decisions are made in classification tasks. Using a variety of input variables, we used Decision Trees to model the decision-making process for identifying websites as either real or phishing [22]. In our implementation, we utilized the `DecisionTreeClassifier` class from the scikit-learn library, to classify websites as phishing or legitimate. It is important to highlight here that, although this model can effectively capture complex patterns in data, is very prone to overfitting.

Random Forest: Random Forest is a versatile ensemble learning algorithm that combines multiple decision trees for better predictive accuracy and in controlling overfitting. Each tree in the forest is trained on a random subset of the data, and its predictions are aggregated to determine the final output, typically through majority voting for classification tasks or averaging for regression tasks [23].

The main advantage of using Random Forest is the fact that it can handle large datasets with higher dimensionality and that it is very effective in estimating missing data. Also, it reduces overfitting by averaging out multiple decision trees, hence more reliable predictions [24].

In our implementation, we have used `RandomForestClassifier` from the scikit-learn library. It provides an efficient and robust way to carry out classification tasks. This classifier builds multiple decision trees during training and outputs the class that is the mode of classes predicted by individual trees [25].

4.1.3.3. Clustering Methods

To find patterns and structure in the dataset, clustering methods were used in addition to classification techniques. These unsupervised learning methods are especially helpful for exploratory data analysis, which aims to group similar data points based on their feature space without the need for predefined labels.

K-means Clustering: As the primary clustering technique, the K-means Clustering algorithm was employed. This algorithm is a straightforward iterative clustering algorithm. Using the distance as the metric and given the K classes in the data set, calculate the distance mean, giving the initial centroid, with each class described by the centroid [26]. The approach is useful for finding cohesive groupings in the dataset because it iteratively reduces the within-cluster variation.

In our implementation, the number of clusters (k) was determined using the Elbow Method, which evaluates the sum of squared distances (inertia) to assess the balance between model complexity and performance. To complement K-Means, Hierarchical Clustering was also applied. This method creates a dendrogram, showcasing the nested structure of clusters at varying levels of granularity. In order to calculate the number of clusters (k), we implemented the elbow method, which assesses the balance between model complexity and performance by calculating the sum of squared distances (inertia) [27].

Agglomerative Clustering: Following the K-means Clustering algorithm, we implemented agglomerative clustering to find hierarchical categories in the dataset. This technique is a member of the hierarchical clustering algorithm family, which uses a bottom-up strategy to create clusters. Each data point is initially created as a separate cluster, then they are gradually combined based on how similar they are. Also, it is important to mention that we used the Ward's Method, which produces compact and spherical clusters by minimising the variance of the clusters being merged.

4.1.4. Incorporating Neural Networks

Incorporating neural networks into phishing detection analysis can significantly enhance the system's ability to identify complex patterns associated with phishing attacks. It is important to mention that neural networks, have demonstrated superior performance in classification tasks due to their ability to model complex, non-linear relationships within the data [4].

Multilayer Perceptrons (MLPs): MLPs can be viewed as generalizations of linear models that perform multiple stages of processing to reach a decision. Including the usage of the Multilayer Perceptron in phishing detection systems improved the results regarding malicious website detection significantly. MLPs, a class of feedforward artificial neural networks, have been found effective in modeling complicated nonlinear relationships inherent in data; therefore, they are suitable for classification of legitimate or phishing sites. Indeed, the studies conducted in this regard show that MLP-based models outperform some traditional classifiers like K-Nearest Neighbors, Support Vector Machines, and Decision Trees on different metrics [5].

Perceptron Classifier Model: In our approach, the Perceptron Classifier was used-a basic binary classifier that serves as a basis for numerous neural networks. The Perceptron algorithm works by modifying its weights in a way to reduce its classification errors iteratively. This makes it effective on linearly separable data. Its simplicity and efficiency make it valuable for preliminary modeling and understanding of the linear separability of the dataset. However, the Perceptron can learn only linear boundaries, and complex nonlinear distributions of data may be challenging for it. However, this simple model provides an insight into the structure of the data and is usually the first step toward more sophisticated models [24]. Regarding our implementation, a wide range of hidden layer structures was explored, including single-layer networks (e.g., (32,)), (64,)) as well as multi-layer networks with deeper architectures (e.g., (32, 16), (64, 32, 16)). As for the activation functions, two were tested: tanh and ReLu activations, while the Adam optimizer was used for the training process.

XGBoost Classification Model: XGBoost (eXtreme Gradient Boosting) is a gradient boosting algorithm created for efficiency and effectiveness. It functions by forming a group of decision trees, with each new tree aiming to fix the mistakes of the earlier ones, thereby enhancing the model's predictive ability. Key features of XGBoost include regularization techniques to prevent overfitting, parallel processing capabilities for efficient computation as well as the ability to handle missing values gracefully. These attributes make XGBoost very efficient in classification problems, such as the detection of phishing websites, where pattern complexity and the interaction between those patterns are essential [31]. In our analysis, we used this algorithm to improve the accuracy of detection by effective modeling of the complex relationships that exist between several website features and their likelihood of being associated with phishing activities.

Support Vector Machines (SVMs): SVM is one of the popular supervised machine learning algorithms which have found a broad field of applications right from the general classification of instances to detecting phishing websites. It basically tries to find an optimal hyperplane which maximizes the margin between different classes in high-dimensional space. By projecting the input data onto higher-dimension space using a kernel function, both linear and nonlinear classifications are possible in support vector machines. In practice, to implement SVMs, great care has to be taken while choosing the parameters, such as the kernel function and regularization parameter, to have the best performance. Despite all these challenges, SVMs remain a robust and effective tool in developing phishing detection systems [32].

Autoencoder Neural Networks: Autoencoder neural networks are unsupervised learning models developed to find a data representation method to be employed either for feature learning or dimensionality reduction. Such techniques include a linked encoder, in charge of compressing input to a representation inside a latent space, and an associated decoder tasked with restoring inputs from this new latent representation. An autoencoder for phishing could therefore learn the basic model of the patterns of

an authentic site [33]. In our analysis, the autoencoder was constructed with an encoder to reduce the dimensionality and a decoder to reconstruct the output. It was trained to minimize the reconstruction error by comparing the original input and the reconstructed output. Also, it is important to highlight that a loss function, mean squared error (MSE) was used, along with two different optimizers, tanh and relu.

4.1.5. Evaluation Metrics

In every machine learning assessment, the evaluation metrics are crucial for measuring the efficiency and dependability of the models analyzed. These metrics offer a numerical insight into the performance of the models on the classification task, allowing for a structured comparison of their advantages and disadvantages. In this research, where the main objective is to detect phishing websites, selecting the right evaluation metrics guarantees that both prediction accuracy and the costs associated with possible errors are properly considered.

The evaluation metrics used in this analysis include:

Accuracy: Measures the proportion of correctly predicted instances over the total number of instances. While useful, accuracy alone can be misleading in the presence of imbalanced datasets [31].

Precision: This represents the fraction of true positives among all the positive predictions by the model. High precision is a representation that a model can avoid false positives. It is important to prevent such a model from misclassifying legitimate websites as phishing in order not to have a negative effect on users' experience [31].

Recall: This is the ratio of the true positives returned to the total number of actual positives. High recall guarantees that the model will catch phishing websites and hence few false negatives [31].

F1-score: The harmonic mean of precision and recall. It gives a balanced measure of the model's performance. It is useful in cases just like in phishing detection tasks, where the precision and recall values need to be balanced [31].

Confusion Matrix: A matrix that encapsulates the effectiveness of a machine learning model evaluated on a specific set of test data. It provides the count of instances generated by the model on the test data, including the true and false positives alongside the true and false negatives, thus offering more profound insights into the model's classification mistakes [32].

Area Under the Receiver Operating Characteristic Curve (AUC-ROC): AUC-ROC is one of the most useful metrics in machine learning for binary classification models. It

essentially describes the capability of a model to differentiate between positive and negative classes over a variety of threshold settings. The AUC value varies from 0 to 1, with values closer to 1 indicating better model performance. This metric is specifically useful in cases of imbalanced datasets, as it provides a wider scope than considering the accuracy of the model itself. [31]

The ROC curve itself is a graphical plot that shows the true positive rate (sensitivity) against the false positive rate (1 - specificity) at various threshold settings. AUC conveys the probability that a randomly selected positive example is ranked higher by the model than a randomly selected negative one. This probability view again emphasizes the appropriateness of AUC as a ranking measure. [33]

Loss Curves: Loss curves are one of the most important tools in machine learning since they allow the representation of a model performance during training visually. By plotting the loss value against the number of training iterations or epochs, they give an idea of how well a model is learning over time. The analysis of these curves can therefore be helpful in diagnosing problems such as overfitting or underfitting. A big difference between training and validation loss could indicate overfitting, when it performs well during training but generalizes poorly. Convergence at high loss in both cases would be evidence of underfitting-the model is too simple and fails to understand the underlying pattern within the data. Understanding loss curves and how to interpret them correctly is essential to attaining optimum results from the models and, indeed, effectively teaching them [34].

Silhouette Score: The silhouette score, in general, is a metric of the quality of clustering in data analysis. It tells how much an object resembles the cluster it's in as opposed to other clusters. It has a range between -1 and +1. With a higher silhouette score, it denotes that the object is well matched to its own cluster and poorly matched to the neighbor cluster, showing better-defined clusters. This metric is widely used for both the determination of the optimal number of clusters in a dataset and the validity assessment of clustering. Works have shown its application in image segmentation and deep clustering methods [35, 36].

t-Distributed Stochastic Neighbor Embedding (t-SNE): t-SNE is a method for reducing dimensionality that translates high-dimensional datasets into a lower-dimensional format while maintaining their fundamental structure. It accomplishes this by transforming pairwise similarities among data points in the high-dimensional space into joint probabilities, subsequently reducing the divergence between these probabilities in both high and low-dimensional spaces. This method successfully preserves local relationships in the data, which makes t-SNE especially valuable for detecting clusters and revealing patterns [37].

5. Results

5.1. Statistical Summary of Features

The dataset presents a relatively balanced distribution between phishing and legitimate websites, minimizing the potential for model bias. Preliminary analysis highlights that specific features, such as `having_IP_Address`, `SSLfinal_State`, and `Request_URL`, exhibit a strong correlation with the target variable. Additionally, features like `age_of_domain` and `Page_Rank` reveal significant interrelationships and noteworthy associations with the target variable.

To further explore the dataset, the following visualizations were generated:

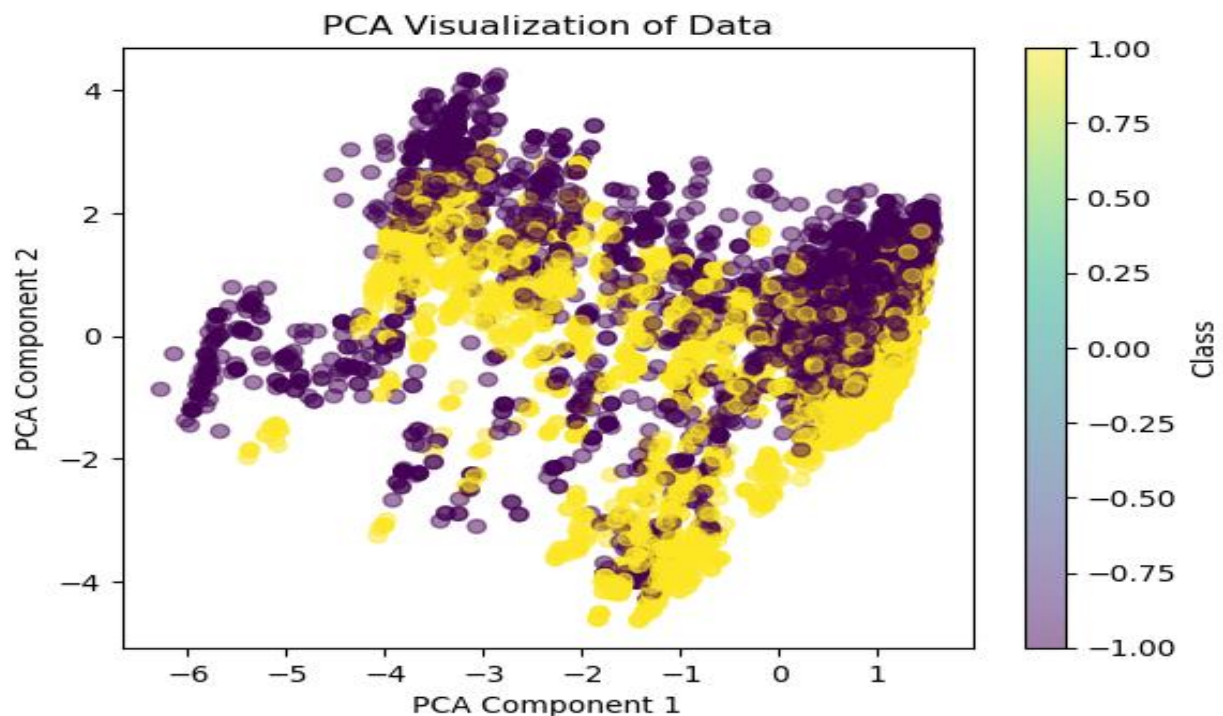


Figure 1:Original Dataset Visualization with PCA

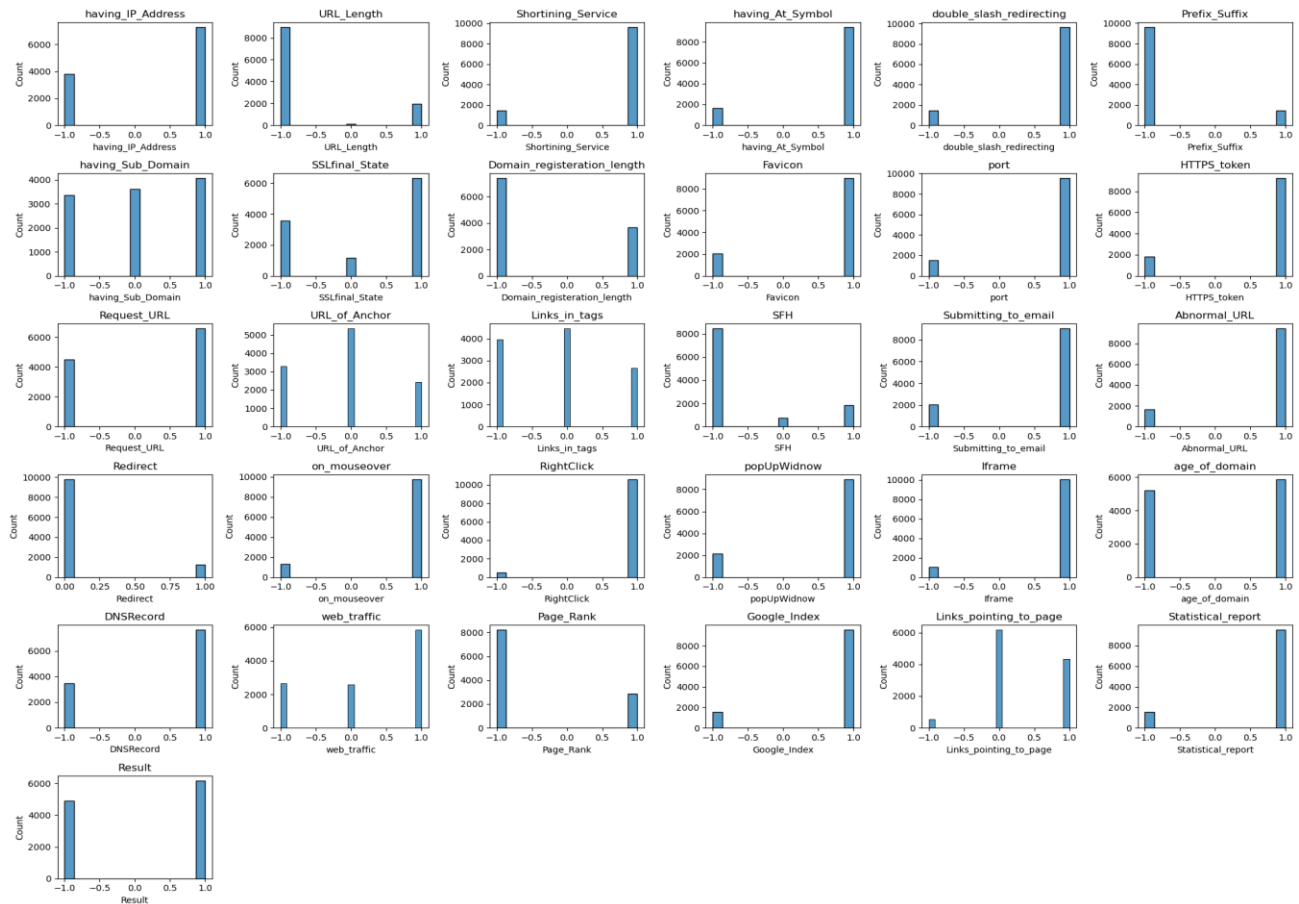


Figure 2: Data Distribution among Features

Up close the distribution of the target variable “Result”:

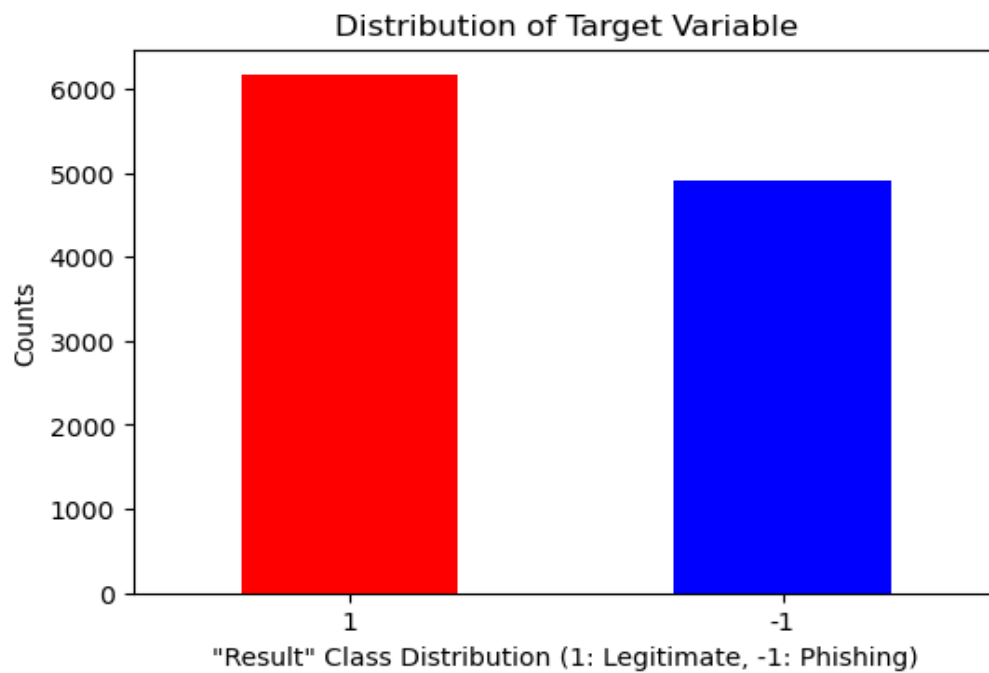


Figure 3: Distribution of Target Variable "Result"

The following analysis examines the distribution of phishing (Class -1) and legitimate (Class 1) websites across different values of key features. This information helps in identifying features with significant discriminatory power between the two classes. In more detail, each feature's value in the following table affect whether a URL is more likely to belong to class -1 (phishing) or class 2 (legitimate). By looking at the percentages for each feature's value across the classes, we are able to see how strongly each feature of the dataset influences the classification process.

Feature	Feature Value	Class -1 (%)	Class 1 (%)
having_IP_Address	-1	50.78	49.22
	1	40.93	59.07
URL_Length	-1	45.52	54.48
	0	61.48	38.52
	1	37.55	62.45
Shortining_Service	-1	35.60	64.40
	1	45.61	54.39
having_At_Symbol	-1	50.57	49.43
	1	43.20	56.80
double_slash_redirecting	-1	39.33	60.67
	1	45.04	54.96
Prefix_Suffix	-1	51.07	48.93
	1	0.00	100.00
having_Sub_Domain	-1	54.59	45.41
	0	62.18	37.82
	1	19.90	80.10
SSLfinal_State	-1	85.77	14.23
	0	98.20	1.80
	1	11.07	88.93
Domain_registration_length	-1	36.41	63.59
	1	60.23	39.77
Favicon	-1	44.28	55.72
	1	44.34	55.69
port	-1	48.87	51.13
	1	43.59	56.41
HTTPS_token	-1	39.81	60.19
	1	45.18	54.82
Request_URL	-1	59.51	40.49
	1	33.89	66.11
URL_of_Anchor	-1	98.90	1.10
	0	28.14	71.86
	1	6.16	93.84
Links_in_tags	-1	60.34	39.66
	0	39.20	60.80
	1	28.94	71.06
SFH	-1	50.21	49.79
	0	34.56	65.44
	1	21.41	78.59
Submitting_to_email	-1	46.23	53.77

	1	43.88	56.12
Abnormal_URL	-1	37.08	62.92
	1	45.55	54.45
Redirect	0	43.94	56.06
	1	47.07	52.93
on_mouseover	-1	49.96	50.04
	1	43.54	56.46
RightClick	-1	47.27	52.73
	1	44.12	55.83
popUpWidnow	-1	44.31	55.69
	1	44.30	55.70
Iframe	-1	43.77	56.23
	1	44.36	55.64
age_of_domain	-1	50.72	49.28
	1	38.63	61.37
DNSRecord	-1	49.90	50.10
	1	41.78	58.22
web_traffic	-1	63.01	36.99
	0	66.87	33.13
	1	25.84	74.16
Page_Rank	-1	47.37	52.63
	1	35.49	64.51
Google_Index	-1	60.23	39.77
	1	41.73	58.27
Links_pointing_to_page	-1	35.22	64.78
	0	47.58	52.42
	1	40.82	59.18
Statistical_report	-1	54.13	45.87
	1	42.70	57.30

Table 3: Class Distribution Analysis Across Feature Values

For features of three values, the value of 0 is usually used to represent a neutral, middle or undefined state. We can also learn from Class -1 vs. Class 1 distribution at value 0, the cases in which a feature has an insignificant or ambiguous effect on the classification. Examples include; a high Class -1 percentage for the 0 value may indicate that the absence or neutrality of a feature still contributes to the phishing class. On the other hand, if the 0 value has a relatively balanced distribution across Class -1 and Class 1, it may point toward the reduced import of the feature for the classification process.

Since there are percentages assigned to every value (-1, 0, 1), analysis of the influence of every feature upon the classification process is allowed, including the role that neutral or undefined states take to define the class.

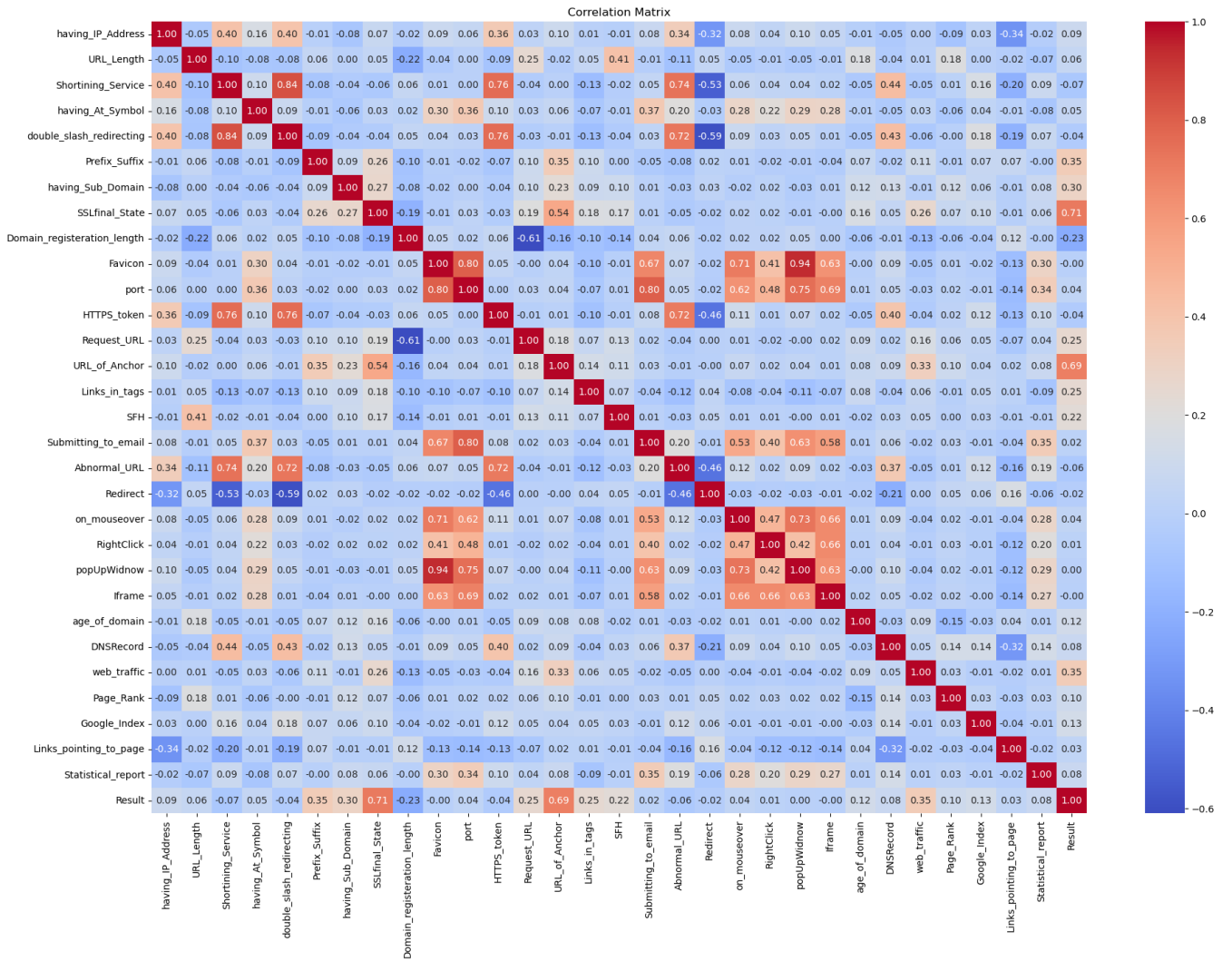


Figure 4: Feature Correlation Matrix

The correlation matrix presented above offers a comprehensive summary of the linear connections among features in the dataset. Every cell shows the Pearson correlation coefficient, which spans from -1 (indicating a perfect negative correlation) to +1 (indicating a perfect positive correlation). Features that exhibit strong correlations (whether positive or negative) may suggest possible redundancy or dependencies, which are crucial factors for feature selection and model development.

Some key observations from the above correlation matrix:

- The features SSLfinal_State and Result are highly positively correlated, which means they vary highly directly. This could mean that SSLfinal_State is a good predictor of the target variable.
- Request_URL and Result are highly negatively correlated, which shows that with lower values of Request_URL, the websites are phishing (Class -1).

- Features such as URL_of_Anchor and Links_in_tags are highly inter-correlated. This might indicate redundancy and could even be subject to dimensionality reduction or feature selection techniques.
- Features related to Prefix_Suffix and Domain_registration_length exhibit meaningful correlations with Result, which shows their importance in the classification task.
- Specific features, such as Favicon and Iframe, have very poor correlations with the target variable; such attributes may indicate low contribution to prediction towards the target class.

Below we will present the results regarding the correlation of features with the target variable. The correlations have been computed in two ways to give a broader understanding of feature relevance. One sorted by absolute value and the other based on their actual values.

→ Correlation of features with the target variable (sorted by absolute value):

Result	1.000000
SSLfinal_State	0.714741
URL_of_Anchor	0.692935
Prefix_Suffix	0.348606
web_traffic	0.346103
having_Sub_Domain	0.298323
Request_URL	0.253372
Links_in_tags	0.248229
Domain_registration_length	0.225789
SFH	0.221419
Google_Index	0.128950
age_of_domain	0.121496
Page_Rank	0.104645
having_IP_Address	0.094160
Statistical_report	0.079857
DNSRecord	0.075718
Shortining_Service	0.067966
Abnormal_URL	0.060488
URL_Length	0.057430
having_At_Symbol	0.052948
on_mouseover	0.041838
HTTPS_token	0.039854
double_slash_redirecting	0.038608
port	0.036419
Links_pointing_to_page	0.032574
Redirect	0.020113
Submitting_to_email	0.018249
RightClick	0.012653
Iframe	0.003394
Favicon	0.000280
popUpWidnow	0.000086

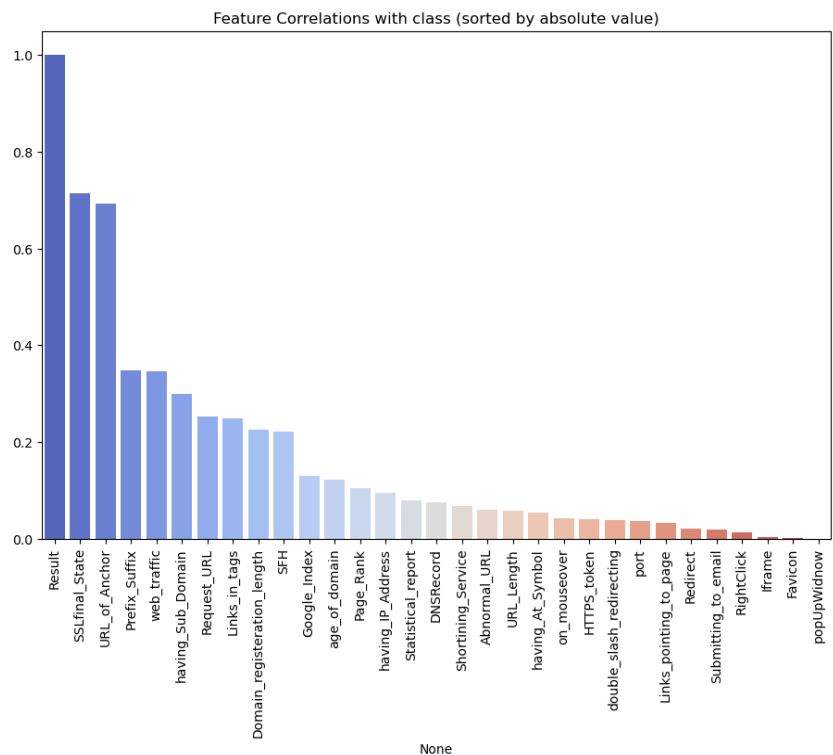


Figure 5: Feature Correlations with class (absolute values)

→ Correlations with the target variable 'Result' (actual values):

Result	1.000000
SSLfinal_State	0.714741
URL_of_Anchor	0.692935
Prefix_Suffix	0.348606
web_traffic	0.346103
having_Sub_Domain	0.298323
Request_URL	0.253372
Links_in_tags	0.248229
SFH	0.221419
Google_Index	0.128950
age_of_domain	0.121496
Page_Rank	0.104645
having_IP_Address	0.094160
Statistical_report	0.079857
DNSRecord	0.075718
URL_Length	0.057430
having_At_Symbol	0.052948
on_mouseover	0.041838
port	0.036419
Links_pointing_to_page	0.032574
Submitting_to_email	0.018249
RightClick	0.012653
popUpWidnow	0.000086
Favicon	-0.000280
Iframe	-0.003394
Redirect	-0.020113
double_slash_redirecting	-0.038608
HTTPS_token	-0.039854
Abnormal_URL	-0.060488
Shortining_Service	-0.067966
Domain_registration_length	-0.225789

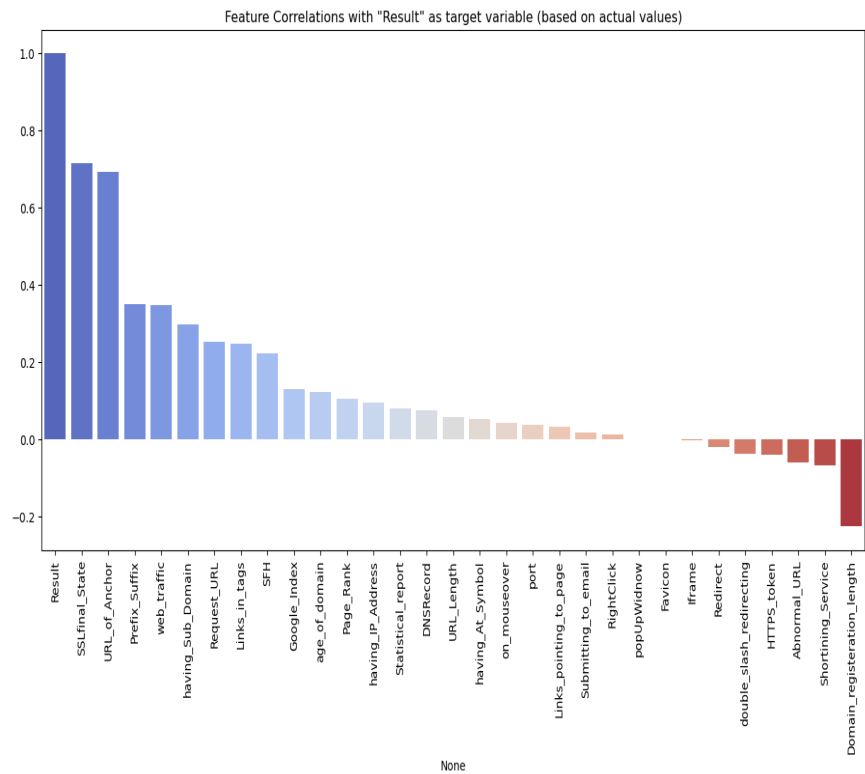


Figure 6: Feature Correlations with class (actual values)



Figure 7: Correlation Analysis of Features with the Target Variable

Following the analysis we did based on the correlation matrix and the feature's correlation values; we implement a threshold of value 0.2 in order to get the most relevant features of the dataset. These relevant features will be our "dataset 1".

- Dataset 1: (Most relevant features retrieved based on threshold > 0.2)

Relevant features:	
Result	1.000000
SSLfinal_State	0.714741
URL_of_Anchor	0.692935
Prefix_Suffix	0.348606
web_traffic	0.346103
having_Sub_Domain	0.298323
Request_URL	0.253372
Links_in_tags	0.248229
SFH	0.221419
Domain_registration_length	-0.225789

Table 4: Dataset 1

Utilizing Dataset 1, which includes the most significant features selected through the correlation threshold (> 0.2), we then proceeded to evaluate the different machine learning models. The aim of this stage is to evaluate the effectiveness of these features in the classification process and to identify the best model for forecasting phishing websites. By focusing on the most relevant features in Dataset 1, we simply aim to simplify the model while retaining its predictive power.

After splitting the dataset into training and testing sets using an 80/20 split, we standardized the features to ensure they were on similar scale and implemented the following classification algorithms: Logistic Regression, KNN, Decision Tree Classifier and Random Forest Classifier. In order to evaluate the performance for each classification model, we utilized the following key metrics: accuracy, precision, recall, F1-score and the confusion matrix.

The results obtained from this analysis are presented altogether in the following figure (Figure 8).

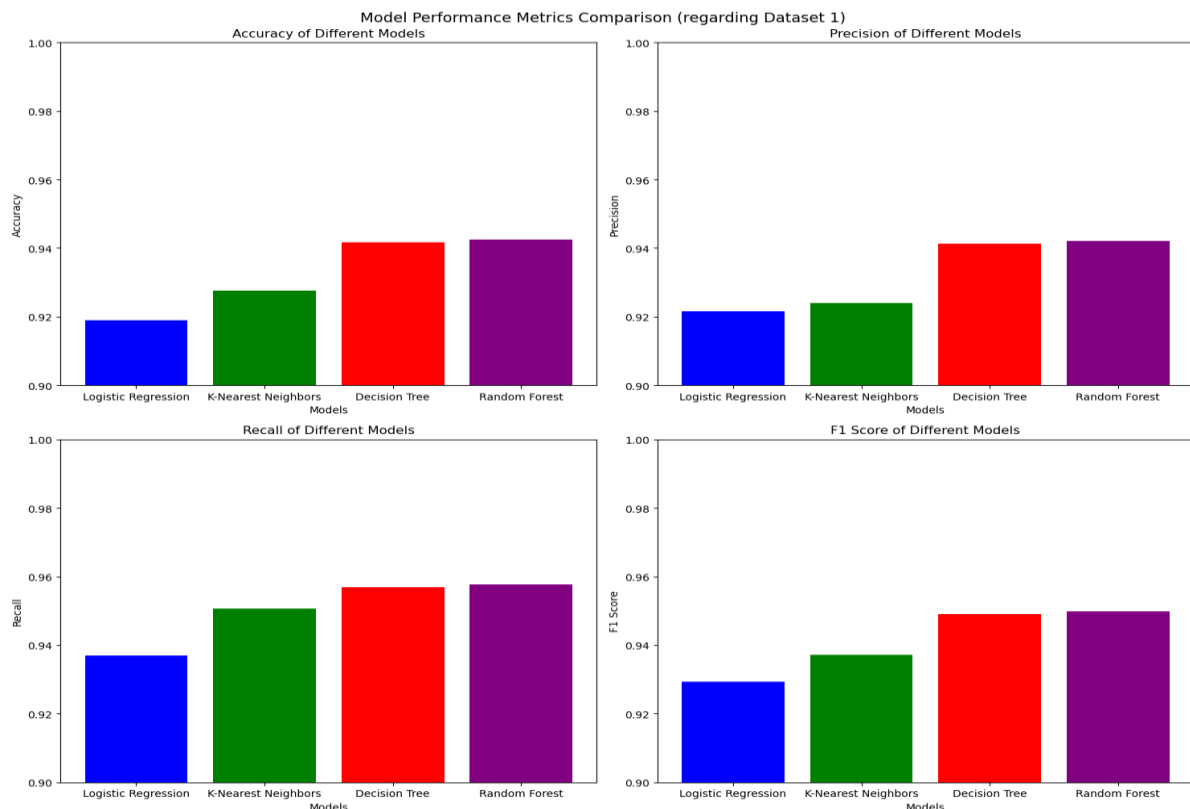


Figure 8: Model Performance Metrics Comparison (Dataset 1)

Following the above, we will now create a second dataset (Dataset 2), by employing an alternative feature selection approach. Specifically, we apply in our main original dataset, the Sequential Feature Selector (SFS) to select the best subset of

features for classification. This subset will be the Dataset 2. This refined dataset will be used to evaluate the impact of advanced feature selection techniques. The same classification algorithms as well as evaluation metrics, as before, will be applied in order to ensure consistency and comparability with the results we gathered from Dataset 1.

Dataset 2: (Most relevant features retrieved from SFS)

Selected Features with SFS
having_IP_Address
URL_Length
having_At_Symbol
Prefix_Suffix
SSLfinal_State
port
URL_of_Anchor
Submitting_to_email
Statistical_report
Result

Table 5: Dataset 2

The results obtained from this analysis are presented altogether in the following figure (Figure 9).

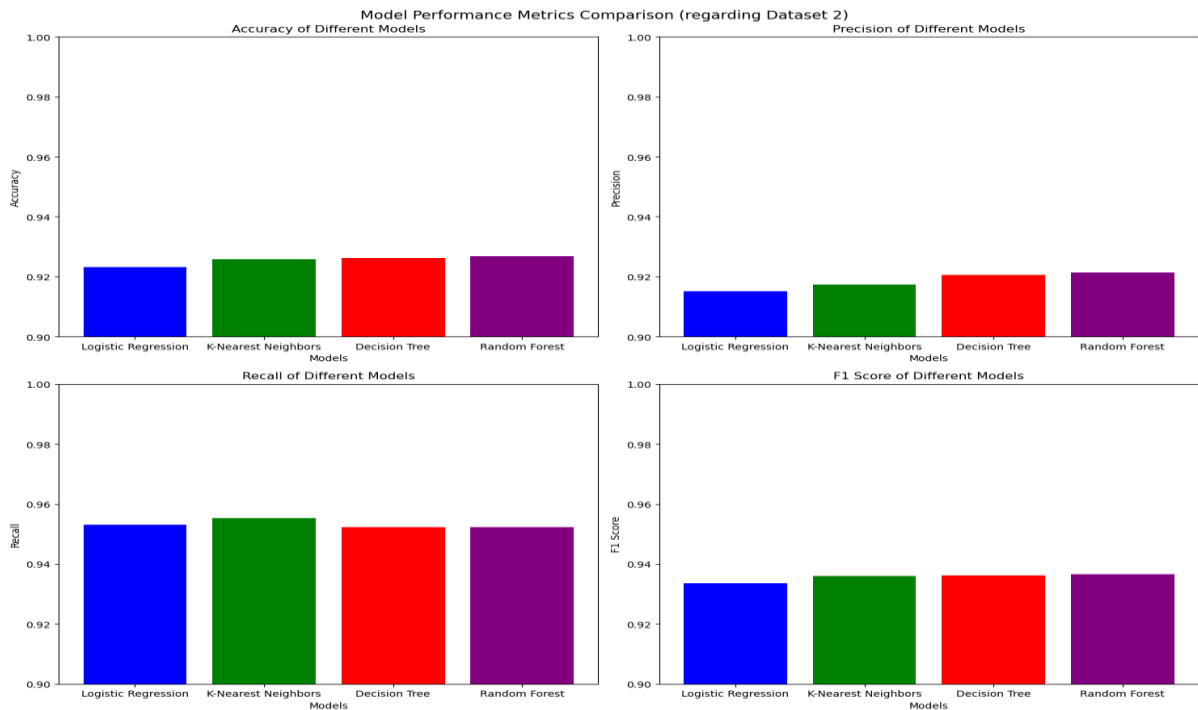


Figure 9: Model Performance Metrics Comparison (Dataset 2)

Based on the above figure (Figure 9), Decision Tree and Random Forest performed well in all measures, depicting their robustness and efficiency for the phishing

detection task. Logistic Regression gives the lowest scores but still reliably performs as a baseline model. K-Nearest Neighbors is balanced in its performance above Logistic Regression but slightly lower than Decision Tree and Random Forest.

Following the implementation of the SFS, we now proceed to perform the Recursive Feature Elimination (RFE) on the selected features. This subset will be the Dataset 3. The same classification algorithms as well as evaluation metrics, as before, will be applied in order to ensure consistency and comparability with the results we gathered from Dataset 1. It is important to mention though, that the K-Nearest Neighbors model cannot be used with RFE, so we will exclude it in this step.

Dataset 3: (Most relevant features retrieved from RFE)

Selected Features with RFE
having_IP_Address
Prefix_Suffix
having_Sub_Domain
SSLfinal_State
URL_of_Anchor
Links_in_tags
SFH
Web_traffic
Google_index
Result

Table 6: Dataset 3

The results obtained from this analysis are presented altogether in the following figure (Figure 10).

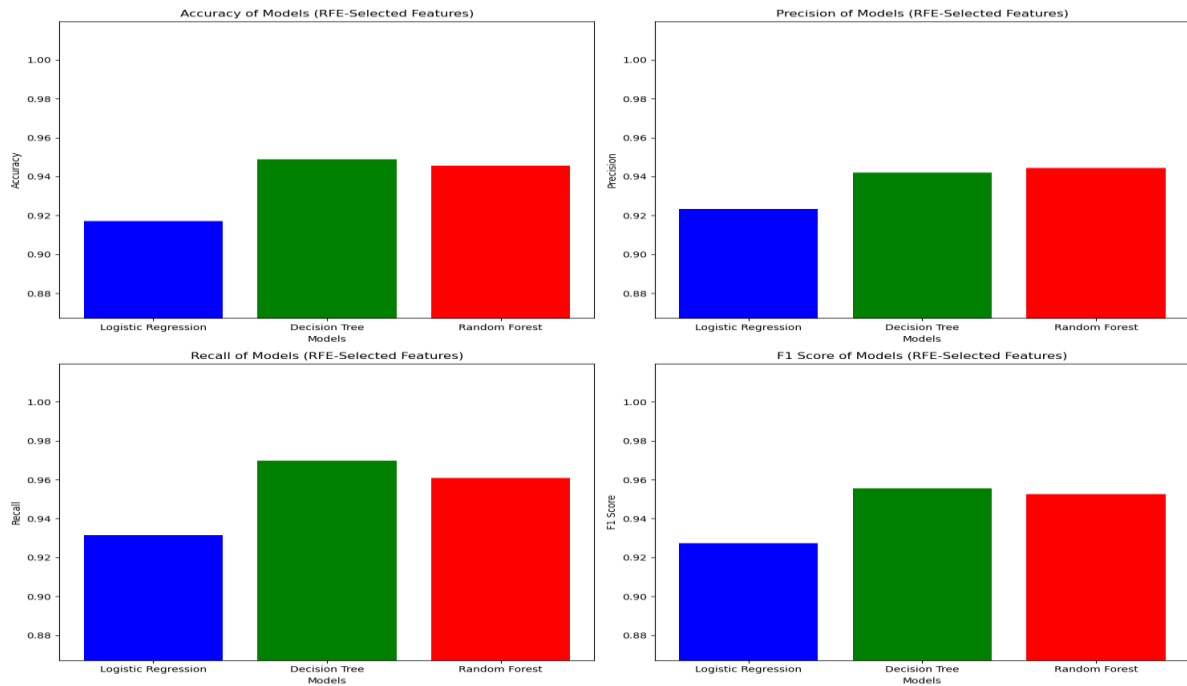


Figure 10: Model Performance Metrics Comparison (Dataset 3)

Based on the above figure (Figure 10), we can see that the Decision Tree and Random Forest models continue to outperform Logistic Regression across all metrics, showcasing their effectiveness when using RFE-selected features. Logistic Regression performs reliably but lags behind the other models.

Feature Importance:

Understanding feature importance is a fundamental step in the performance and interpretability of any machine learning model. Feature importance helps to identify which features are most useful for the model to make predictions, refine the dataset, improve model efficiency, and enhance decision-making. The importance of the features derived from various classification models developed in this analysis are also considered here. The observation of an importance score would provide a better view of what factors are important in the classification of phishing sites. These values are visualized to show the contributions of each feature across different models, thus giving a comprehensive understanding of their role in classification. This step lays the foundation for further refinement and optimization of the models.

The results obtained, for each different model used, are presented below.

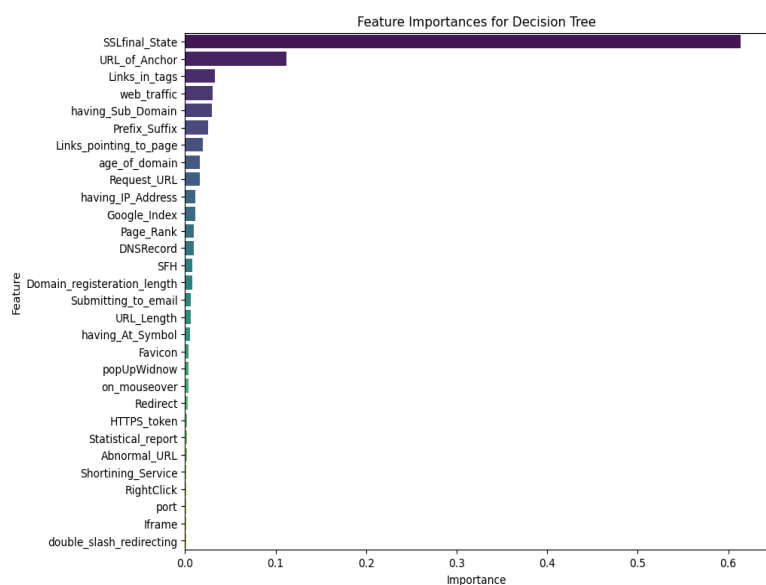


Figure 12: Feature Importance for Decision Tree Classifier

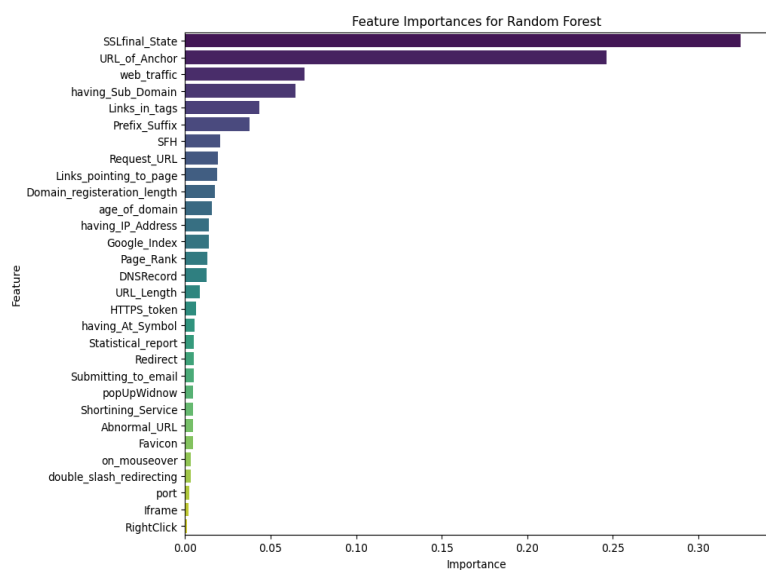


Figure 11: Feature Importance for Random Forest Classifier

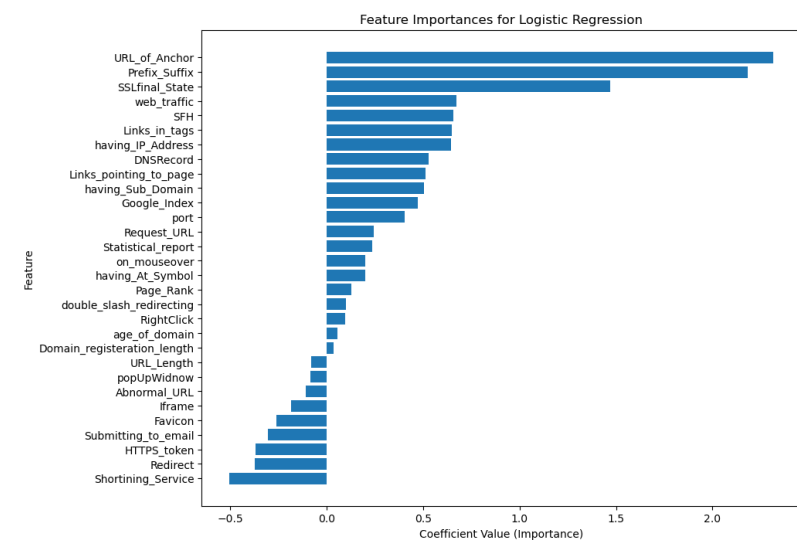


Figure 13: Feature Importance for Logistic Regression Classifier

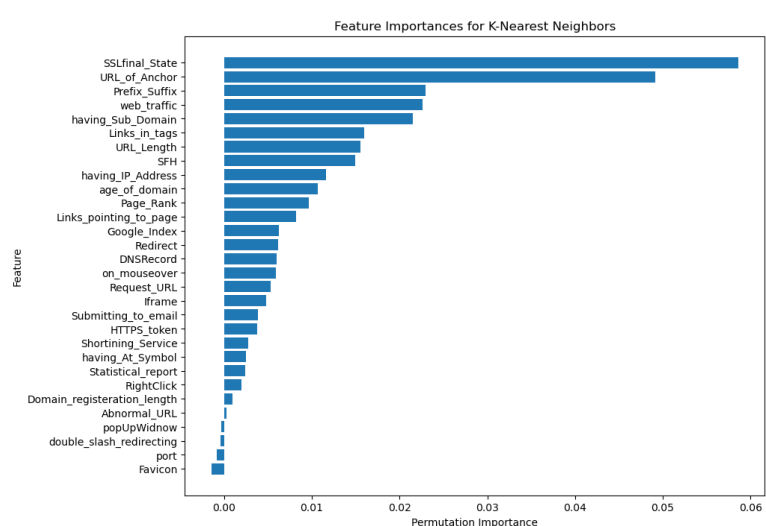


Figure 14: Feature Importance for KNN Classifier

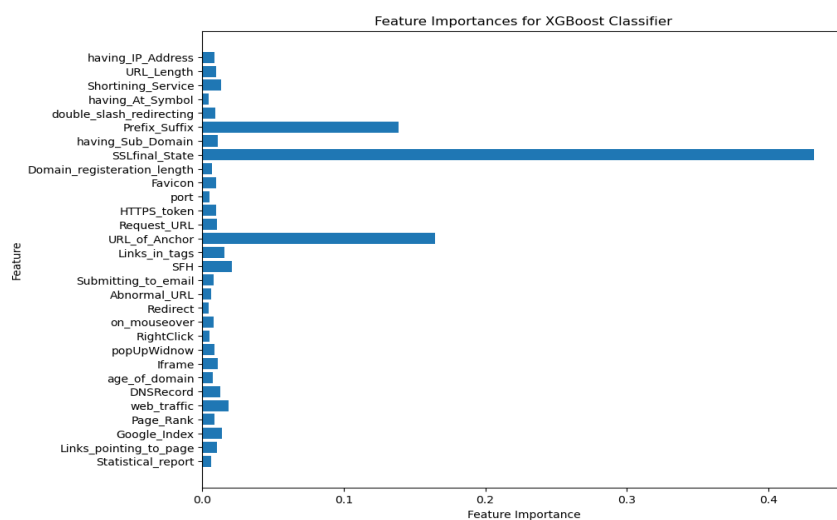


Figure 15: Feature importance for XGBoost Classifier Model

Clustering Analysis:

Clustering analysis forms the basis of understanding the structure of the dataset and estimating the capability of models to effectively group similar data points. While classification focused on predicting pre-defined labels, clustering is about finding patterns intrinsic to the data, and thus it helps in providing an insight into the separability of phishing and legitimate websites.

Our implementation shows the clustering potential of the different models applied to the dataset. We visualize data distribution and assess the quality of clustering by silhouette scores that can give us an idea of how well the models can tell apart the two classes. Cluster compactness and separation are evaluated with techniques such as k-means clustering and t-SNE visualizations.

K-means Clustering:

k-values	Silhouette score
k=2	0.284
k=3	0.284
k=4	0.113
k=5	0.107
k=6	0.106
k=7	0.106
k=8	0.091
k=9	0.085

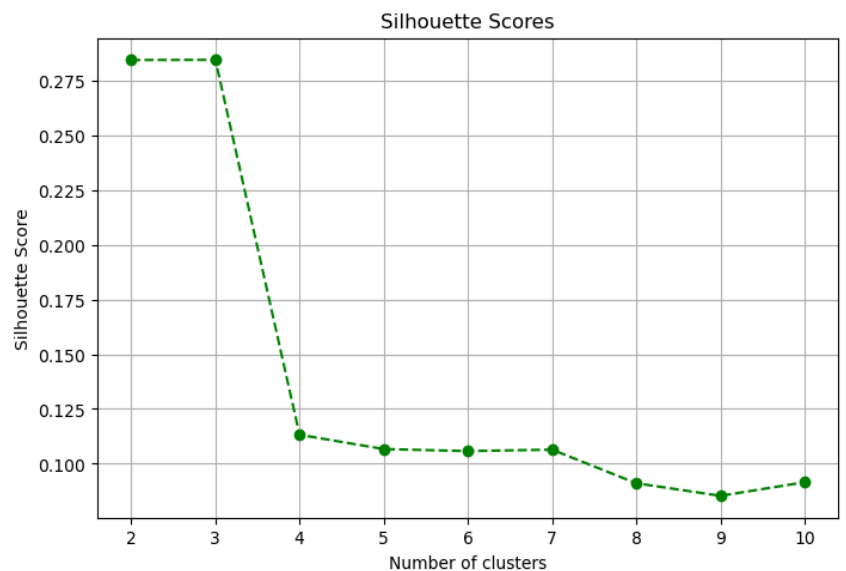


Figure 16: Silhouette Scores (K-means Clustering)

We performed clustering to identify the best number of clusters by applying the silhouette score. After running various values of k, the silhouette analysis provided 3 clusters as the best for clustering performance with the highest silhouette score of 0.284. It followed that each of the data points needed a cluster label attached, and that should become an included feature within the dataset. In so doing, it transformed the dataset to a shape of (11,055, 31) - an updated shape containing really good information. Furthermore, based on the elbow method in the plot below (Figure 16), we can see a distinct “elbow” at k=3, suggesting this as the optimal number of clusters.

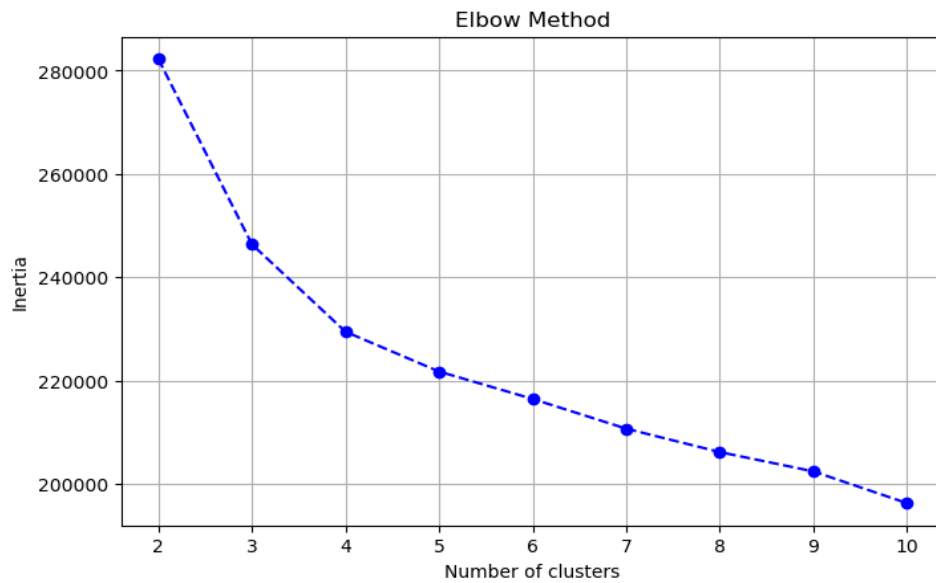


Figure 17: Elbow Method (k-means)

Following, we evaluated the impact of Clusters in the k-means by implementing the following classification algorithms: Logistic Regression, KNN, Decision Tree Classifier and Random Forest Classifier. In order to evaluate the performance for each classification model, we utilized the following key metrics: accuracy, precision, recall, F1-score and the confusion matrix. We also visualized the clustering results of the k-means algorithm. Each color in the visualization represents a distinct cluster, highlighting this way the separability of data point into three main groups.

The vizualization results are presented below (Figure 18).

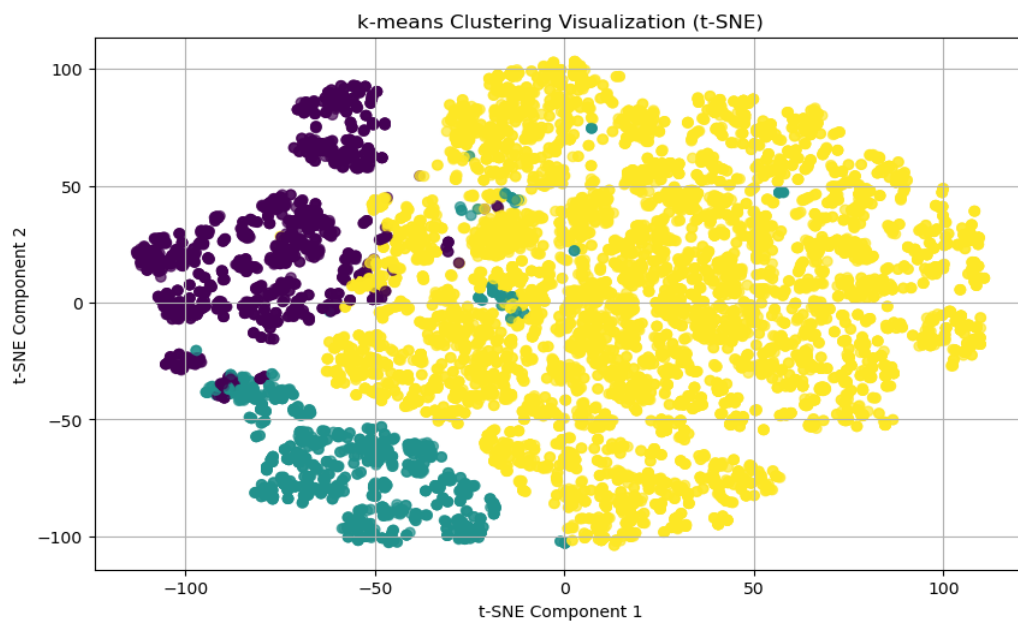


Figure 18: k-means Clustering Visualization

Model	Accuracy (%)	Precision (Weighted Avg)	Recall (Weighted Avg)	F1-Score (Weighted Avg)
Random Forest	96.6	0.97	0.97	0.97
Decision Tree	95.6	0.96	0.96	0.96
Logistic Regression	92.2	0.92	0.92	0.92
K-Nearest Neighbors (KNN)	93.2	0.93	0.93	0.93

Table 7: Model Evaluation Results (k-means Clustering)

Some key points regarding the above table (Table 7):

- The Random Forest Classifier demonstrated the best overall performance with the highest accuracy and balanced precision, recall, and F1-score.
- Regarding the Decision Tree Classifier, it also performed well but with slightly lower accuracy and F1-scores.
- The Logistic Regression model, while simpler, showed competitive performance, especially in precision and recall.
- Lastly, regarding the KNN classifier, it showed robust performance, outperforming Logistic Regression but slightly lagging behind the ensemble methods.

Agglomerative Clustering:

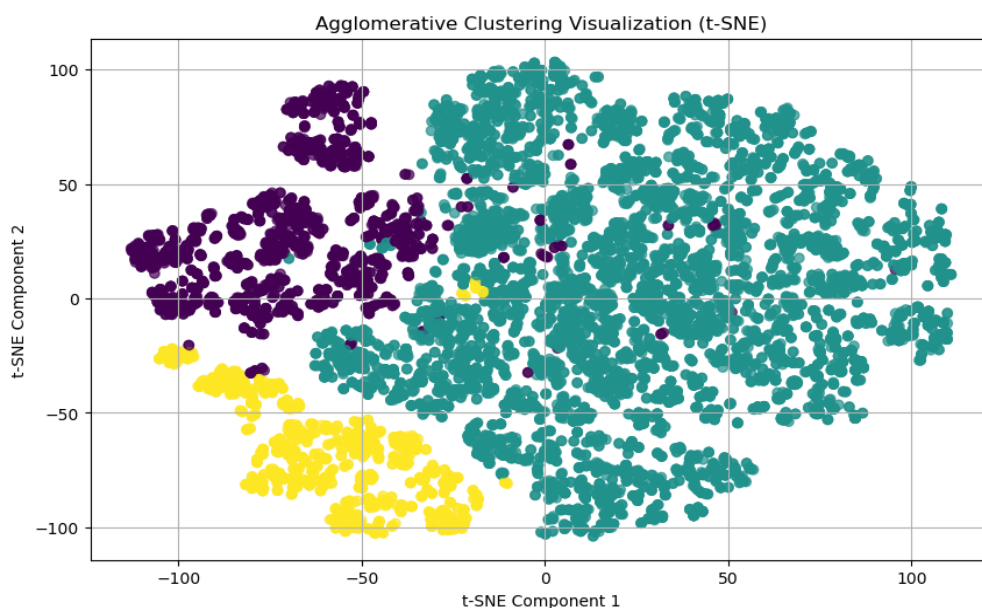


Figure 19: Agglomerative Clustering Visualization

Model	Accuracy (%)	Precision (Weighted Avg)	Recall (Weighted Avg)	F1-Score (Weighted Avg)
Random Forest	96.8	0.97	0.97	0.97
Decision Tree	95.7	0.96	0.96	0.96
Logistic Regression	92.1	0.92	0.92	0.92
K-Nearest Neighbors (KNN)	93.1	0.93	0.93	0.93

Table 8: Model Evaluation Results (Agglomerative Clustering)

Some key points regarding the above table (Table 8):

- Random Forest performed best overall, with the highest accuracy and very strong precision, recall, and F1-scores.
- Decision Tree returned similar results while maintaining a high accuracy but was more interpretable than ensemble methods.
- Logistic Regression, being a simpler linear model, performed decently and thus can be used for baseline comparisons.
- KNN showed strong results, closely following Random Forest in both accuracy and F1-scores.

Neural Network Analysis

As we mentioned before, neural networks represent a powerful model of machine learning. Traditional models lack this ability, and because of this reason, neural networks usually perform very well in tasks for which there are intricate feature interactions, such as the detection of phishing websites. This section discusses the analysis done for neural networks regarding their implementation and performance in classifying either phishing or legitimate websites.

Multilayer Perceptrons (MLPs):

We begin our analysis with arguably the simplest of neural networks: the Multilayer Perceptrons. In MLPs, fully connected layers process data through multiple hidden layers, each further equipped with an activation function to introduce non-linearity in the data. We study hidden layer configurations, types of activation functions, and various optimization techniques in search of the most suitable architecture that will best fit this classification task.

Following, the results obtained for the different hidden layers are presented.

Configuration info: {'hidden_layer_sizes': (32,), 'activation': 'tanh', 'solver': 'adam', 'max_iter': 1000}
Accuracy: 0.9669832654907282

Configuration info: {'hidden_layer_sizes': (64,), 'activation': 'tanh', 'solver': 'adam', 'max_iter': 1000}
Accuracy: 0.9678878335594754

Configuration info: {'hidden_layer_sizes': (32, 16), 'activation': 'tanh', 'solver': 'adam', 'max_iter': 1000}
Accuracy: 0.9633649932157394

Configuration info: {'hidden_layer_sizes': (64, 32), 'activation': 'tanh', 'solver': 'adam', 'max_iter': 1000}
Accuracy: 0.966078697421981

Configuration info: {'hidden_layer_sizes': (64, 32, 16), 'activation': 'tanh', 'solver': 'adam', 'max_iter': 1000}
Accuracy: 0.9633649932157394

Configuration info: {'hidden_layer_sizes': (32,), 'activation': 'relu', 'solver': 'adam', 'max_iter': 1000}
Accuracy: 0.9665309814563546

Configuration info: {'hidden_layer_sizes': (64,), 'activation': 'relu', 'solver': 'adam', 'max_iter': 1000}
Accuracy: 0.9678878335594754

Configuration info: {'hidden_layer_sizes': (32, 16), 'activation': 'relu', 'solver': 'adam', 'max_iter': 1000}
Accuracy: 0.9592944369063772

Configuration info: {'hidden_layer_sizes': (64, 32), 'activation': 'relu', 'solver': 'adam', 'max_iter': 1000}
Accuracy: 0.9601990049751243

Configuration info: {'hidden_layer_sizes': (64, 32, 16), 'activation': 'relu', 'solver': 'adam', 'max_iter': 1000}
Accuracy: 0.9642695612844867

Loss Curves for Each Configurations using tanh as Activation (MLP) and Adam as Solver

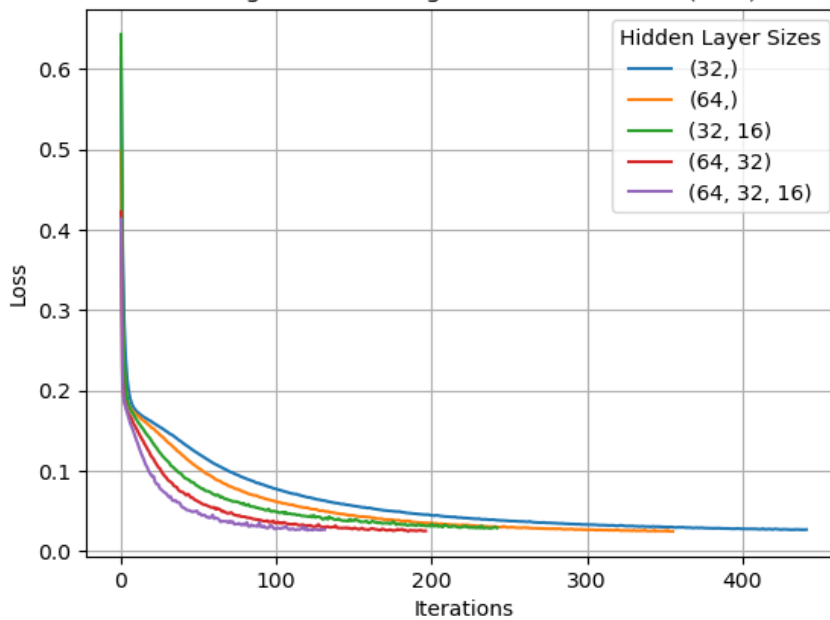


Figure 20: Loss Curves for MLP Configurations (activation:tanh, solver:adam)

Loss Curves for Each Configurations using ReLu as Activation (MLP) and Adam as Solver

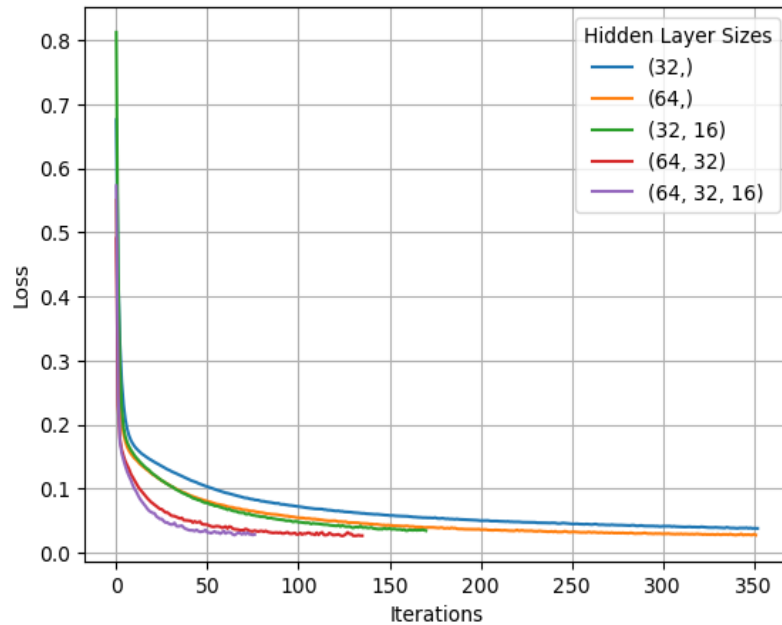


Figure 21: Loss Curves for MLP Configurations (activation:ReLu, solver:adam)

After trying different configurations of the Multilayer Perceptron, we have identified the best architecture for this phishing detection task. The configuration **hidden_layer_sizes=(64,)** with **tanh** as the activation function and **Adam** as the **optimizer** has shown the best overall performance regarding convergence speed and loss minimization, as can be seen in the loss curves.

The choice of selecting tanh as the activation function over ReLU was made simply because of the consistency that had been given. Though ReLU is a pretty standard choice for models due to low computational costs, tanh seemed much more adequate here, because training results were quite steady and predictable under this type.

Further on, we trained and checked the best configuration on the MLP model on the prepared dataset. This one included training with a maximum of 1,000 iterations to make sure it converged, using random_state=42 for its reproducibility. In this case, the precision, recall, accuracy, and F1-score tests were good configurations since it performed relatively well in the task of classification, thereby being robust when it comes to phishing detection.

Below, we present the results obtained from the training and the evaluation of the best configuration.

Model Accuracy on Test Set: 0.9678878335594754

Confusion Matrix:

```
[[ 910  46]
 [  25 1230]]
```

Classification Report:

	precision	recall	f1-score	support
-1	0.97	0.95	0.96	956
1	0.96	0.98	0.97	1255
accuracy			0.97	2211
macro avg	0.97	0.97	0.97	2211
weighted avg	0.97	0.97	0.97	2211

Experimenting with Different Training Sizes and Evaluating the Results

In order to evaluate the model's strength and applicability, we performed experiments with different sizes of training sets. This analysis aims to assess the impact of training data volume on the model's performance and to determine the ideal equilibrium between training size and accuracy.

→ The original dataset was split into training and testing subsets with different training set proportions (10%, 20%, 30%, 40%, 50%, 60%, 70%, 80% and 90% training data) and each training set size was compared with its corresponding test set in order to ensure a more consistent evaluation.

→ Regarding the model, we used the MLPs model with the best configuration results: hidden_layer_sizes=(64,) with tanh as the activation function and Adam as the optimizer.

Below, we present the results obtained from the different training sizes.

Training Size (%)	Accuracy	Precision	Recall	F1-Score
10%	0.928	0.934	0.939	0.936
20%	0.942	0.942	0.956	0.949
30%	0.950	0.956	0.955	0.955
40%	0.958	0.957	0.969	0.963
50%	0.963	0.960	0.975	0.967
60%	0.967	0.964	0.979	0.971
70%	0.968	0.969	0.975	0.972
80%	0.968	0.964	0.980	0.972
90%	0.972	0.970	0.981	0.975

Table 9: Summary of Performance Metrics by Training Size

Confusion Matrices

Training Size: 10%

Predicted	Class -1	Class 1
Actual: -1	4027	371
Actual: 1	341	5211

Training Size: 20%

Predicted	Class -1	Class 1
Actual: -1	3594	294
Actual: 1	216	4740

Training Size: 30%

Predicted	Class -1	Class 1
Actual: -1	3206	191
Actual: 1	197	4145

Training Size: 40%

Predicted	Class -1	Class 1
Actual: -1	2740	163
Actual: 1	116	3614

Training Size: 50%

Predicted	Class -1	Class 1
Actual: -1	2271	128
Actual: 1	79	3050

Training Size: 60%

Predicted	Class -1	Class 1
Actual: -1	1813	93
Actual: 1	53	2463

Training Size: 70%

Predicted	Class -1	Class 1
Actual: -1	1369	59
Actual: 1	48	1841

Training Size: 80%

Predicted	Class -1	Class 1
Actual: -1	910	46
Actual: 1	25	1230

Predicted	Class -1	Class 1
Actual: -1	462	19

Training Size: 90%

Actual: 1	12	613
------------------	----	-----

Model Performance with Different Training Sizes (MLP)

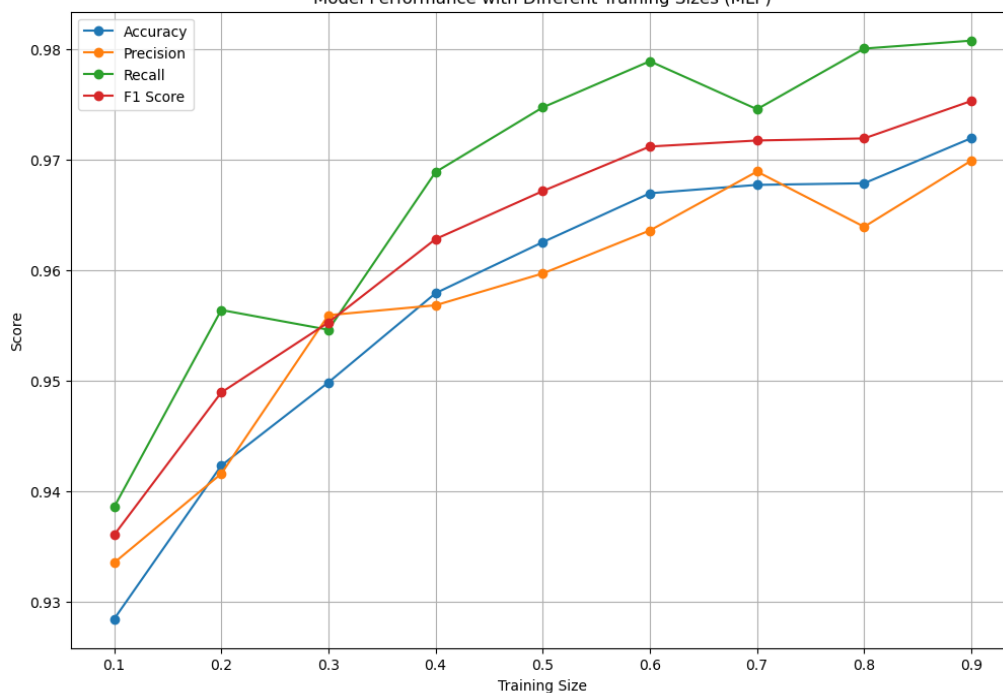


Figure 24: Model Performance for the Different Training Sizes

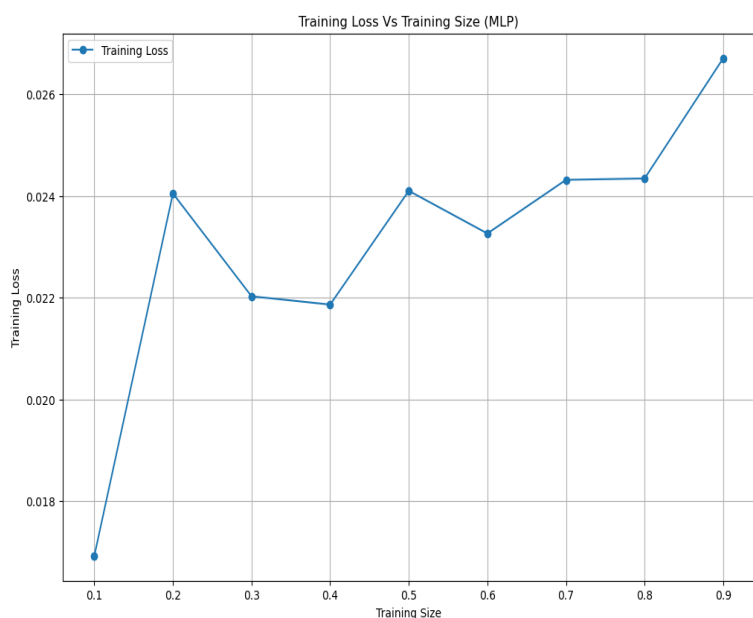


Figure 23: Training Loss Vs Training Size (MLP)

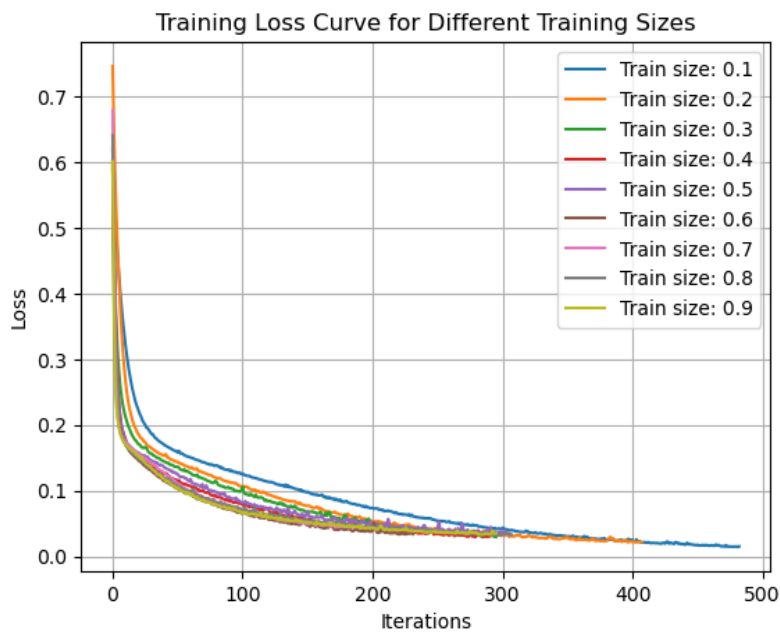


Figure 22: Training Loss Curve for the Different Training Sizes

Above are the plots showing the relationship of training loss vs. training size and training loss curves at different training sizes over 500 iterations for the MLP model.

In the left plot (Figure 22), we can see that there is a slight fluctuation in training loss while increasing the training size. Whereas with the more minor training sizes, like 0.1, the training loss remains low, likely due to the simpleness of subsets that the model gets trained on, in the case of larger train size, it goes up slightly to reflect that this model has seen harder patterns and a greater variety. Despite these ups and downs, training loss is persistently low throughout all the sizes, hence reflecting that the model learned well from the data.

The right plot (Figure 23) shows the training loss curves for different train sizes. All these curves show a rapid convergence in the first couple of iterations where the loss was steeply decreased. Larger sets, such as 0.7, 0.8, and 0.9, converge a bit slowly compared to other smaller sets, which agrees with what one would expect. However, the final loss values across all training sizes are almost the same, which further speaks to the robustness and adaptability of the model.

These visualizations confirm that the MLP model generalizes well, maintains low loss, and balances training size with computational efficiency for the classification task.

Perceptron Classification Model:

As we mention earlier, the perceptron model offers important insights into classification and acts as a standard for assessing more intricate models. In the subsequent section, we examine how well the Perceptron classification model performs on the phishing detection dataset.

Below are the results we retrieved from this analysis.

Accuracy of the Perceptron model: 0.907

Classification Report for the Perceptron model:

	precision	recall	f1-score	support
-1	0.89	0.90	0.89	956
1	0.92	0.91	0.92	1255
accuracy			0.91	2211
macro avg	0.90	0.91	0.91	2211
weighted avg	0.91	0.91	0.91	2211

Confusion Matrix (Perceptron):

```
[[ 859  97]
 [ 109 1146]]
```

The Perceptron classification model gave an accuracy of 90.7%, which shows that despite the simplicity of the algorithm, it performed well on the phishing detection task. The classification report is also well balanced, having almost similar precision, recall, and F1-scores for both classes, with Class -1 (phishing) giving a precision of 89% and Class 1 (legitimate) giving a precision of 92%.

Its confusion matrix represents the classification of most of the instances correctly, while the model mislabeled 97 instances as false positives-legitimate websites that were classified as phishing-and 109 instances as false negatives-actual phishing websites which were predicted as legitimate.

Therefore, these results suggest that Perceptron indeed performs well as a linear model. However, it could be somewhat incapable with regard to complicated patterns in the dataset.

XGBoost Classification Model:

In the following section, we analyze the performance of the XGBoost classification model on the phishing detection dataset, highlighting its strengths, key metrics, and effectiveness in comparison to other models.

The results obtained from its implementation are the following:

XGBoost: Accuracy on training Data: 0.989

XGBoost : Accuracy on test Data: 0.971

Precision: 0.965

Recall: 0.984

F1 Score: 0.974

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.95	0.97	956
1	0.96	0.98	0.97	1255
accuracy			0.97	2211
macro avg	0.97	0.97	0.97	2211
weighted avg	0.97	0.97	0.97	2211

Confusion Matrix:

```
[[ 911  45]
 [  20 1235]]
```

The XGBoost classification model outperforms all other techniques, yielding 98.9% and 97.1% performance for the training and test data, respectively, considering that generalization has been pretty good. The precision was 96.5%, recall 98.4%, with an

F1-score of 97.4%, describing the performance of the model in classifying phishing and legitimate websites.

From the classification report, the performance is relatively balanced between the two classes: Class 0, representing phishing, has a recall of 95%, while Class 1, representing legitimate data, has a recall of 98%. The confusion matrix presents a very low level of misclassification, having only 45 false positives and 20 false negatives.

These results highlight that XGBoost effectively captured the complex patterns involved and is reliable for the phishing detection task.

Support Vector Machines (SVM):

In this section, we analyze the performance of the SVM classification model on the phishing detection dataset, highlighting its strengths, key metrics, and effectiveness in comparison to other models. For this analysis, we included a comprehensive comparison of different kernel functions such as linear, polynomial (poly), radial basis function (RBF) and sigmoid in order to determine the kernel best suited for this classification task. Also, for each kernel, we measured the Mean Squared Error Value.

Kernel	Accuracy (%)	Precision	Recall	F1-Score	Mean Squared Error
Linear	92.85	0.93	0.93	0.93	0.2858
Polynomial (Poly)	95.02	0.95	0.95	0.95	0.1990
RBF	95.25	0.95	0.95	0.95	0.1899
Sigmoid	82.95	0.83	0.83	0.83	0.6820

Table 10: SVM Model Performance with Different Kernels

Confusion Matrices:

Kernel: linear

	Predicted: -1	Predicted: 1
Actual: -1	861	95
Actual: 1	63	1192

Kernel: Polynomial (poly)

	Predicted: -1	Predicted: 1
Actual: -1	882	74
Actual: 1	36	1219

Kernel: RBF

Kernel: Stigmoid

	Predicted: -1	Predicted: 1
Actual: -1	887	69
Actual: 1	36	1219

	Predicted: -1	Predicted: 1
Actual: -1	771	185
Actual: 1	192	1063

Comparison of Neural Network Models

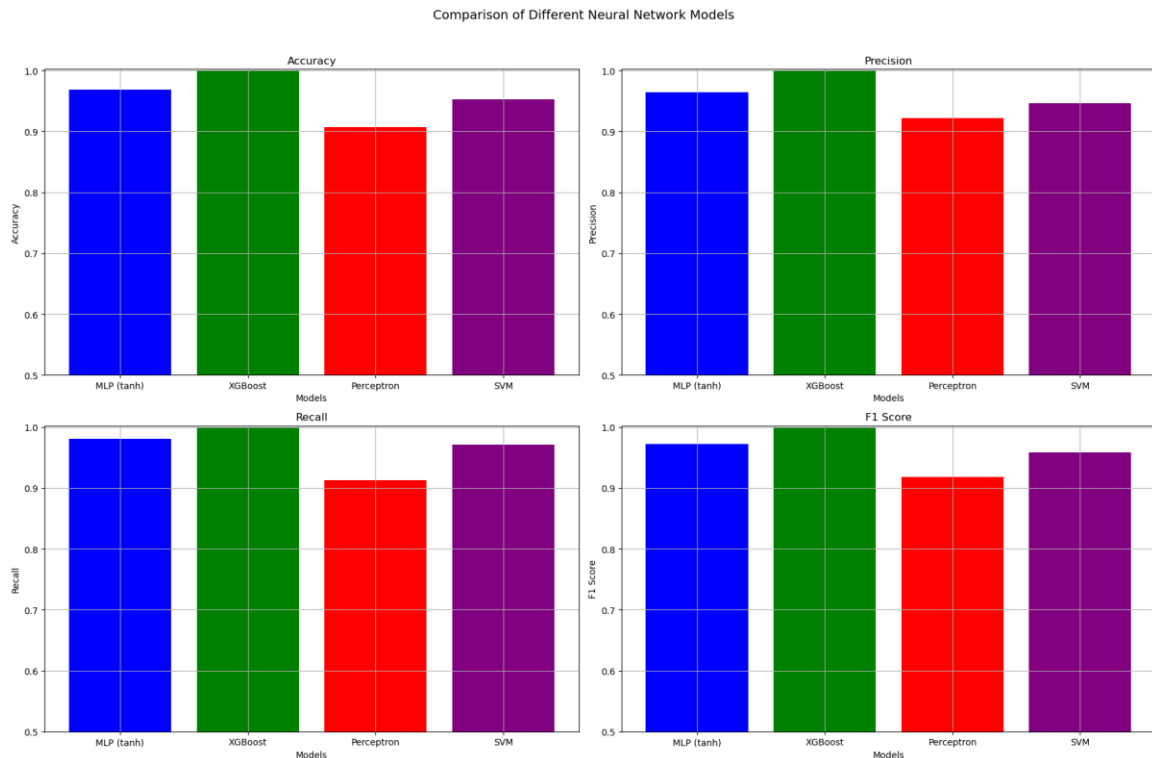


Figure 25: Comparisson of Neural Network Models

In the figure above (Figure 25) we can see that XGBoost and SVM represent the best value among all four metrics. Very close to this top performance result, the model MLP (tanh) presents also a very good behavior. Perceptron results are not that far, with respect to the competitiveness of the score. This is an indication of how effective the advanced models of neural networks were for this classification task.

Autoencoder Neural Network:

An Autoencoder Neural Network was put in place as part of the final analysis to explore its capability for feature extraction and dimensionality reduction. As we already know, autoencoders are unsupervised learning models for data compression to a lower-dimensional representation and then its reconstruction. While capturing the most important patterns in the data, the Autoencoder helps identify latent features that can enhance the performance of a classification model.

This approach was used to discover hidden structures in the dataset and assess its potential contribution to the accuracy of phishing detection.

Also, it is important to mention that we designed and evaluated two autoencoder models, utilizing **tanh** and **ReLU** activation functions, and analyzed their performance by visualizing the training and validation loss curves.

Here are the visualization plots that we got.

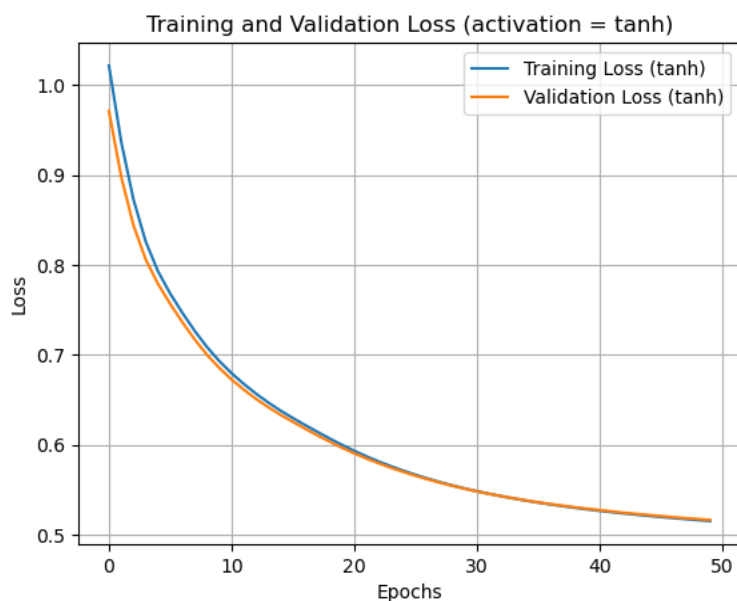


Figure 27: Training and Validation Loss (activation:tanh)

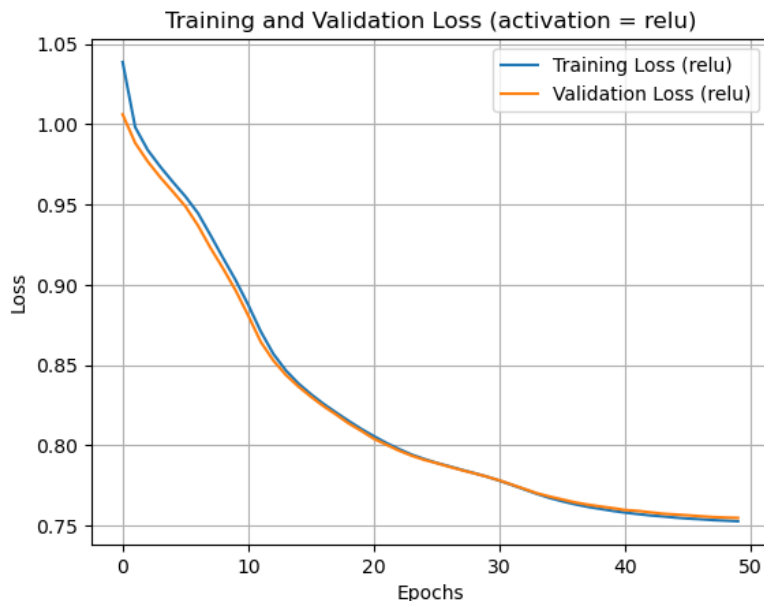


Figure 26: Training and Validation Loss (activation:ReLU)

Encoded training data shape (tanh):(8844,7)
Encoded test data shape (tanh):(2211,7)

Encoded training data shape (relu):(8844,7)
Encoded test data shape (relu):(2211,7)

Comparing the classification models by using ROC and calculating the AUC

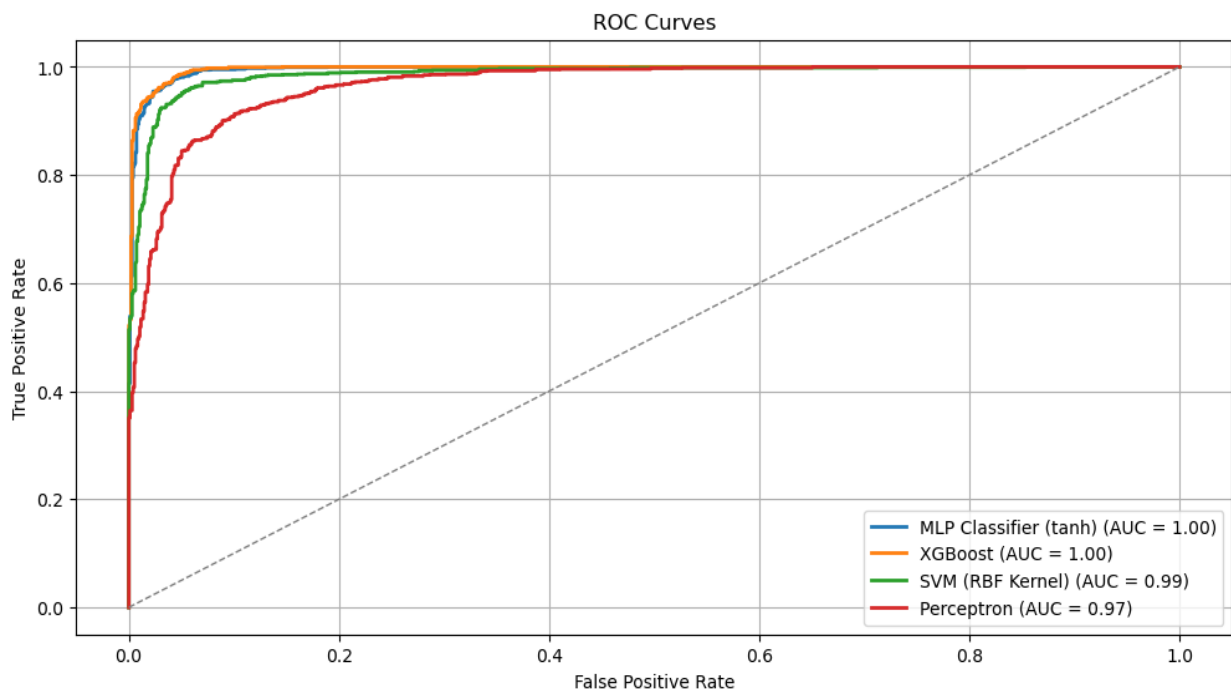


Figure 28: ROC Curves & AUC Values for the different Classification Models

Overfitting Check

The last step of our analysis is to conduct an overfitting analysis to ensure the robustness and generalizability of the models.

Below are the results we obtained regarding the training and test accuracies of the different classification models.

	Model	Training Accuracy	Test Accuracy
1	XGBoost	0.988919	0.970602
0	MLP	0.989824	0.967888
2	SVM	0.959634	0.952510
3	Perceptron	0.907508	0.906829

Table 11: Training and Test Accuracies of NN models

→ The results are sorted first by the values of 'Test Accuracy' and then 'Training Accuracy'

→ For the above comparison, it is clear that the XGBoost Classifier works well with this dataset whilst the MLP follows with very close results.

6. Discussion

6.1. Interpretation of Results and Their Implications

The obtained results from this work prove that machine learning models, especially ensemble methods, such as XGBoost and Random Forest, outperform traditional methods for phishing website detection. XGBoost outperformed with the highest accuracy of 97.1%, with robust generalization, thus demonstrating its capacity for modeling complex patterns inherent in phishing detection tasks. The Random Forest model came second, but still with very strong precision and recall metrics. These findings also show the effectiveness of tree-based algorithms when dealing with high-dimensional datasets with complex relationships between features for classification problems.

This has significantly improved model performance and demonstrated that the optimization of features was an essential task. Other influential predictors included SSLfinal_State, URL_of_Anchor, and web_traffic-all these features being strongly correlated with the target variable. These results also underscores the value of dimensionality reduction using techniques such as autoencoding in engineering and refining a dataset for successful and efficient learning.

6.2. Comparison with Previous Research and Theories

The results obtained are in line with previous research that has highlighted the value of feature-based analysis in phishing detection. For example, previous studies have identified URL characteristics, SSL status, and domain-based features as indicative of phishing activities. However, this study goes a step further by showing that more advanced classifiers, such as XGBoost, outperform traditional models like logistic regression or KNN in terms of predictive power and generalization. Furthermore, unsupervised techniques like clustering and PCA provided valuable insight into data structure, an approach that is rarely found in previous research work. Also, it is important to mention that neural networks, mainly the MLP, established their appropriateness in the domain of phishing detection, although their computational demand remains a limitation.

6.3. Explanation of Limitations or Weaknesses

Despite these robust results, there are several limitations of this study worthy of discussion. Although the dataset used in this work is pretty comprehensive, it may not fully represent the new evolving tactics of phishing attackers; hence, the model may not be perfectly applicable to zero-day threats. Furthermore, slight class imbalances, though mitigated, could still be able to influence model biases when set in real-world applications.

6.4. Suggestions for Future Research

With phishing detection systems increasingly deployed in resource-constrained environments, such as mobile devices or IoT systems, advanced models incur significant computational overhead. Although XGBoost and neural network models tend to have better accuracy, the resource consumption is so high that they cannot be put into practical use. Lightweight machine learning algorithms with both efficiency and effectiveness should be developed in future research. The edge computing solutions might also be applied for offloading heavy computations to cloud-based systems, ensuring real-time detection while keeping the consumption of local resources as low as possible. Meeting these challenges will widen the applicability of phishing detection systems in diverse and constrained operational contexts.

7. Conclusion

This research work presented the development of a machine learning-based phishing website detector with a dataset retrieved from the UC Irvine Machine Learning Repository. In this regard, the following algorithms were trained and tested for phishing website detection: Logistic Regression, K-Nearest Neighbors, Decision Trees, Random Forest, Support Vector Machines, and neural networks such as MLPs, Perceptron and XGBoost.

A summary of Main Findings of our analysis is presented below:

- Among them, some of the highly correlated features that came out were SSLfinal_State, URL_of_Anchor, and web_traffic contributing majorly toward target variable classification.
- Advanced feature selection techniques, including Sequential Feature Selection and Recursive Feature Elimination, when applied, greatly improved model performance by being focused on the most relevant attributes.
- The XGBoost classifier, however, showed the best results; 97.1% test set accuracy and an equally impressive F1-score of 97.4%, demonstrating its capabilities to model the complex relationships present within this data with high accuracy.
- The Multilayer Perceptron gave very strong results, particularly with the tanh-based configuration, underscoring once more the importance of model architecture for the detection of phishing.

- It indicated the flexibility of neural networks to fit non-linear patterns, which is important for classifying phishing and legitimate websites.
- Clustering methods such as K-Means and Agglomerative Clustering showed the intrinsic pattern in the data that could support the separability of phishing and legitimate sites. The Silhouette score was also considered to validate the quality of clustering, and k=3 was the best in K-Means.
- The study applied AUC-ROC, F1-score, and confusion matrices to stress that phishing website detection has to be balanced in order to have a minimum number of false positives and false negatives.

This research has pointed out the efficiency of machine learning in catching the dynamic and ever-changing phishing attacks. While XGBoost had a better performance, even the simplest models, such as Logistic Regression, were competitive and may be well adapted for resource-constrained environments. The neural networks, especially MLPs, have promising directions for future research, mainly regarding improvements in adaptability to new phishing strategies

References

- [1] : Merriam-Webster. (n.d.). Phishing. Retrieved January 6, 2025, from <https://www.merriam-webster.com/dictionary/phishing>
- [2]: Mohammad, R., McCluskey, T. L., & Thabtah, F. (2012). An assessment of features related to phishing websites using an automated technique. In *2012 International Conference for Internet Technology and Secured Transactions (ICITST)* (pp. 492–497). IEEE.
- [3]: Fan, C., Chen, M., Wang, X., et al. (2021). A review on data preprocessing techniques toward efficient and reliable knowledge discovery. *Frontiers in Energy Research*. <https://doi.org/10.3389/fenrg.2021.652801>
- [4]: Bu, S. -J., & Cho, S. -B. (2021). Deep character-level anomaly detection based on a convolutional autoencoder for zero-day phishing URL detection. *Electronics*, 10(12), 1492. <https://doi.org/10.3390/electronics10121492>
- [5]: AAI-Ahmadi, S. (2020). PDMLP: Phishing detection using multilayer perceptron. *International Journal of Network Security & Its Applications (IJNSA)*, 12(3). Available at SSRN: <https://ssrn.com/abstract=3624621>
- [6]: Cheah Chang, P. (n.d.). Multi-layer perceptron neural network for improving detection performance of malicious phishing URLs without affecting other attack types classification. *Cyber Fellows, Tandon School of Engineering*.

- [7]: Built In. (n.d.). Step-by-step explanation of principal component analysis. Retrieved January 6, 2025, from <https://builtin.com/data-science/step-step-explanation-principal-component-analysis>
- [8]: Mohammad, R. M., Thabtah, F., & McCluskey, L. (2015). Phishing websites features. *School of Computing and Engineering, University of Huddersfield*.
- [9]: Re3data.org. (n.d.). Repository entry. Retrieved January 6, 2025, from <https://www.re3data.org/repository/r3d100010960>
- [10]: Mohammad, R., McCluskey, T. L., & Thabtah, F. (2012). An assessment of features related to phishing websites using an automated technique. In *2012 International Conference for Internet Technology and Secured Transactions (ICITST)* (pp. 492–497). IEEE.
- [11]: Komorowski, M., Marshall, D. C., Saliccioli, J. D., & Crutain, Y. (2016). Exploratory data analysis. In *Secondary Analysis of Electronic Health Records* (pp. 267–286). Springer, Cham. https://doi.org/10.1007/978-3-319-43742-2_15
- [12]: Displayr. (n.d.). What is a correlation matrix? Retrieved January 6, 2025, from <https://www.displayr.com/what-is-a-correlation-matrix/>
- [13]: Profillidis, V. A., & Botzoris, G. N. (2019). Chapter 5 - Statistical methods for transport demand modeling. In *Modeling of Transport Demand* (pp. 163–224). Elsevier. <https://doi.org/10.1016/B978-0-12-811513-8.00005-4>
- [14]: Fan, C., Chen, M., Wang, X., et al. (2021). A review on data preprocessing techniques toward efficient and reliable knowledge discovery from building operational data. *Frontiers in Energy Research*. <https://doi.org/10.3389/fenrg.2021.652801>
- [15]: Ibrahimi, E., Lopes, M. B., Dharmo, X., et al. (2023). Overview of data preprocessing for machine learning applications in human microbiome research. *Frontiers in Microbiology*. <https://doi.org/10.3389/fmicb.2023.1250909>
- [16]: Pandas. (n.d.). *isnull*. Retrieved January 6, 2025, from <https://pandas.pydata.org/docs/reference/api/pandas.isnull.html>
- [17]: GeeksforGeeks. (n.d.). Understanding feature importance in logistic regression models. Retrieved January 6, 2025, from <https://www.geeksforgeeks.org/understanding-feature-importance-in-logistic-regression-models/>
- [18]: Peng, C. Y. J., Lee, K. L., & Ingersoll, G. M. (2002). An introduction to logistic regression analysis and reporting. *The Journal of Educational Research*, 96(1), 3–14. <https://doi.org/10.1080/00220670209598786>
- [19]: GeeksforGeeks. (n.d.). K-nearest neighbors. Retrieved January 6, 2025, from <https://www.geeksforgeeks.org/k-nearest-neighbours/>

- [20]: Insights2TechInfo. (n.d.). Unveiling the power of K-nearest neighbors in phishing detection. Retrieved January 6, 2025, from <https://insights2techinfo.com/unveiling-the-power-of-k-nearest-neighbors-in-phishing-detection/>
- [21]: GeeksforGeeks. (n.d.). Decision tree. Retrieved January 6, 2025, from <https://www.geeksforgeeks.org/decision-tree/>
- [22]: Towards Data Science. (n.d.). Decision tree classifier explained: A visual guide with code examples for beginners. Retrieved January 6, 2025, from <https://towardsdatascience.com/decision-tree-classifier-explained-a-visual-guide-with-code-examples-for-beginners-7c863f06a71e>
- [23]: GeeksforGeeks. (n.d.). Random forest algorithm in machine learning. Retrieved January 6, 2025, from <https://www.geeksforgeeks.org/random-forest-algorithm-in-machine-learning/>
- [24]: Javatpoint. (n.d.). Machine learning random forest algorithm. Retrieved January 6, 2025, from <https://www.javatpoint.com/machine-learning-random-forest-algorithm>
- [25]: Scikit-learn. (n.d.). RandomForestClassifier. Retrieved January 6, 2025, from <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- [26]: Yuan, C., & Yang, H. (2019). Research on K-value selection method of K-means clustering algorithm. *J*, 2(2), 226–235. <https://doi.org/10.3390/j2020016>
- [27]: GeeksforGeeks. (n.d.). Elbow method for optimal value of K in K-means. Retrieved January 6, 2025, from <https://www.geeksforgeeks.org/elbow-method-for-optimal-value-of-k-in-kmeans/>
- [28]: Zhang, P. G. (2000). Neural networks for classification: A survey. *IEEE*. <https://doi.org/10.1109/5326.897072>
- [29]: GeeksforGeeks. (n.d.). ML | XGBoost (Extreme Gradient Boosting). Retrieved January 6, 2025, from <https://www.geeksforgeeks.org/ml-xgboost-extreme-gradient-boosting/>
- [30]: GeeksforGeeks. (n.d.). Support vector machine algorithm. Retrieved January 6, 2025, from <https://www.geeksforgeeks.org/support-vector-machine-algorithm/>
- [31]: GeeksforGeeks. (n.d.). Metrics for machine learning models. Retrieved January 6, 2025, from <https://www.geeksforgeeks.org/metrics-for-machine-learning-model/>
- [32]: GeeksforGeeks. (n.d.). Confusion matrix in machine learning. Retrieved January 6, 2025, from <https://www.geeksforgeeks.org/confusion-matrix-machine-learning/>
- [33]: MLU Explain. (n.d.). ROC-AUC. Retrieved January 6, 2025, from <https://mlu-explain.github.io/roc-auc/>

[34]: Google Developers. (n.d.). Interpreting loss curves. Retrieved January 6, 2025, from <https://developers.google.com/machine-learning/crash-course/overfitting/interpreting-loss-curves>

[35]: Myagmarsuren, S. (2024). Exploring the use of silhouette score in K-means clustering for image segmentation. *Department of Computer Science, Tianjin University of Education and Technology, Ulaanbaatar Mongolia*.

[36]: Vardakas, G., Papakostas, I., & Likas, A. (2024). Deep clustering using the soft silhouette score: Towards compact and well-separated clusters. arXiv. <https://doi.org/2402.00608>

[37]: van der Maaten, L., & Hinton, G. (2008). Visualization data using t-SNE. *Journal of Machine Learning Research*, 9, 2579–2605.

Tables:

Table 1: Key Features of Phishing Website Detection
Table 2: Phishing Detection Rules and Criteria
Table 3: Class Distribution Analysis Across Feature Values
Table 4: Dataset 1
Table 5: Dataset 2
Table 6: Dataset 3
Table 7: Model Evaluation Results (k-means Clustering)
Table 8: Model Evaluation Results (Agglomerative Clustering)
Table 9: Summary of Performance Metrics by Training Size
Table 10: SVM Model Performance with Different Kernels
Table 11: Training and Test Accuracies of NN models

Figures:

Figure 1: Original Dataset Visualization with PCA
Figure 2: Data Distribution among Features
Figure 3: Distribution of Target Variable "Result"
Figure 4: Feature Correlation Matrix
Figure 5: Feature Correlations with class (absolute values)
Figure 6: Feature Correlations with class (actual values)
Figure 7: Correlation Analysis of Features with the Target Variable
Figure 8: Model Performance Metrics Comparison (Dataset 1)
Figure 9: Model Performance Metrics Comparison (Dataset 2)
Figure 10: Model Performance Metrics Comparison (Dataset 3)
Figure 11: Feature Importance for Random Forest Classifier
Figure 12: Feature Importance for Decision Tree Classifier
Figure 13: Feature Importance for Logistic Regression Classifier
Figure 14: Feature Importance for KNN Classifier
Figure 15: Feature importance for XGBoost Classifier Model
Figure 16: Silhouette Scores (K-means Clustering)

Figure 17: Elbow Method (k-means)
Figure 18: k-means Clustering Visualization
Figure 19: Agglomerative Clustering Visualization
Figure 20: Loss Curves for MLP Configurations (activation:tanh, solver:adam)
Figure 21: Loss Curves for MLP Configurations (activation:ReLU, solver:adam)
Figure 24: Training Loss Curve for the Different Training Sizes
Figure 23: Training Loss Vs Training Size (MLP)
Figure 22: Model Performance for the Different Training Sizes
Figure 25: Comparisson of Neural Network Models
Figure 26: Training and Validation Loss (activation:ReLU)
Figure 27: Training and Validation Loss (activation:tanh)
Figure 28: ROC Curves & AUC Values for the different Classification Models